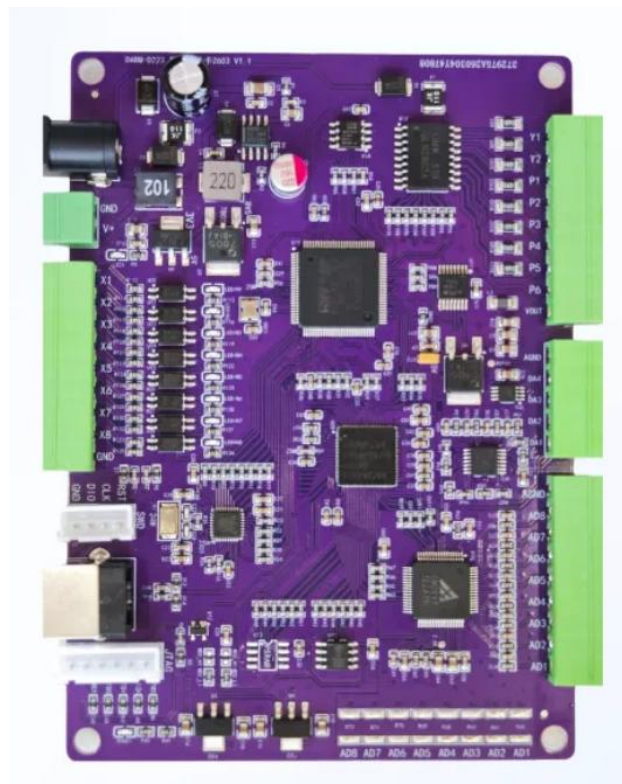


数据采集卡 DABM-D223 用户手册



www.Z-Linear.com

版本: 2605 V1.0

目录

第 1 章 数据采集卡 DABM-D223 说明	1
第 2 章 产品技术参数及功能	2
2.1 模拟输入 (AI)	2
2.2 模拟输出 (AO)	2
2.3 数字输入 (DI)	3
2.4.1 数字输出 (DO)	3
2.4.2 PWM 输出	3
2.5 通信接口	3
2.6 存储系统	3
2.7 电源特性	3
2.8 环境参数	4
2.9 软件支持	4
第 3 章 安装与连接	4
3.1 安装尺寸	4
3.2.1 USB 接口	4
3.2.4 电源系统连接	4
3.3 端子定义及排序说明	5
第 4 章 上位机软件使用	6
4.1 USB 连接	7
6 采集卡记录仪功能介绍	8
6.1 SRAM 记录仪	9
6.3 单点记录仪	10
6.4 多点记录仪	12
6.5 在线记录仪	12
7 采集卡信号源功能	14
7.1 信号源直流功能	14
7.2 正弦波功能	15
7.3 三角波功能	17

7.2 矩形波功能.....	18
7.4 任意波功能.....	20
7.5 信号源标定功能.....	23
8 采集卡 IO 功能.....	28
8.1 采集卡数字输入 DIN 功能	29
8.2 采集卡数字输出 DOUT 功能	30
8.3 采集卡数字输出 PWM 功能	31
9 设置页面	33
9.1 采集卡参数设置.....	34
9.2 采集卡 ADC 校准功能.....	35
10 采集卡 MODBUS RTU 功能	41
11 采集卡 MODBUS TCP 功能	41
12 采集卡 USB 通信协议	41
12.1 C# 上位机接收校验部分代码解析(上位机核心代码).....	41
12.1 单点记录仪	44
12.2 在线记录仪	45
ADC 在线记录仪模式功能使用手册	45
一、功能概述.....	45
二、核心数据结构定义.....	47
1. ADC 发送缓冲区结构体 _adcSendBuf.....	47

第 1 章 数据采集卡 DABM-D223 说明

DABM-D223 是一款集成了 8 路数字输入、8 路数字输出、8 路模拟输入和 4 路模拟输出的常速数据采集卡，可满足模拟量采集、输出、电磁阀、继电器、电机控制等多样化需求。

数据存储方面，DABM-D223 采用 2 种存储解决方案以适配不同的工况要求：1. 高速 8MB 容量的 PSRAM 实时缓存数据，实现采集的波形数据完全记录；2. FRAM 铁电存储器提供非易失性数据存储能力，可以实时缓存编码器、工作状态、设备参数等数据，无需考虑读写寿命。

DABM-D223 使用了双处理器架构进行采集和处理，PSRAM 存储器和 ADC 外接在 FPGA 端，可以实现 ADC 数据全速率搬运存储到 PSRAM 中。USB HS 外挂 STM32H7 侧，借用 ST 的 USB 库，可以简单方便的实现 USB 开发。STM32 和 FPGA 通过 QSPI 接口进行通信，可以同步做到保证速率带宽以及尽量少的占用引脚，方便扩展更多外设。USB HS 通信接口，可以实现 480M 速率和 PC 进行通信，可以方便灵活的对接个系列工控机以及个人电脑。

作为智能制造时代的核心采集单元，DABM-D223 在工业测控场景中展现卓越性能。其微秒级响应速度与多线程处理能力，可同步完成传感器数据采集、执行机构控制及设备状态监测等任务。



扫码进群获取更多资料

第 2 章 产品技术参数及功能

2.1 模拟输入(AI)

- 通道数量: 8 通道单端输入
- 分辨率: 16 位逐次逼近型(SAR) ADC
- 采样速率: 500KSPS, 同步采样
- 模拟带宽: 20K(-3dB), 50K 以内波形可以较完整还原
- 输入范围: $\pm 5V/\pm 10V$
- 输入阻抗: $1M\Omega$
- 硬件精度: $\pm 0.02\%$ (25°C)
- 标定精度: $\pm 0.002\%$ (25°C)
- 2 点标定: 软件可扩展支持, 需用户自备标定源

2.2 模拟输出(AO)

- 通道数量: 4
- 分辨率: 16 位独立基准源专用 DAC
- 输出范围: 0-5V 可编程
- 建立时间: $\leq 0.1ms$
- 输出驱动能力: $\pm 10\text{ mA}$
- 初始精度 $\pm 2.5mV$ (25°C)
- 标定精度 $\pm 1mV$ (25°C)

2.3 数字输入(DI)

- 通道数量: 8 通道非隔离输入
- 逻辑电平: 12-24V
- 硬件响应时间: >100us
-

2.4.1 数字输出(DO)

- 通道数量: 8 通道达林顿开漏输出(需要上拉负载, 如继电器, 光耦等)
- 驱动能力: 100 mA/通道
- 负载电压: 5-24V DC
- 响应时间: $\leq 10 \mu s$
- 过流保护: 板内串接 10Ω 限流电阻

2.4.2 PWM 输出

- 通道数量: 6 路独立 PWM(与 DO 共用输出引脚)
- 频率范围: 1 Hz~5KHz
- 占空比精度: 0~100%可调
- 脉冲数量控制: 有(适合驱动电机脉冲步数)
- 脉冲加减速: 有(适合做电机加减速控制)
- 输出电平: 5V-24V 达林顿开漏输出

2.5 通信接口

- USB: USB 2.0 HS

2.6 存储系统

- 实时缓存: 8MB SRAM
- 非易失存储: 铁电 FRAM, 用于频繁变化数据掉电保存(无限制次读写寿命)

2.7 电源特性

- 供电电压: DC 12-24V 宽压输入

- 电源保护：共模电感, 自恢复保险丝, 防反接二极管, 28V TVS 二极管

2.8 环境参数

- 工作温度：-25℃-85℃（工业级）
- 相对湿度：5%~80%（无凝露）

2.9 软件支持

- 上位机开发环境：C#/QT/labview
- MCU 开发环境：CubeMX, MDK

第 3 章 安装与连接

3.1 安装尺寸

无外壳: 122.4*87.4mm (长宽, 不含连接器)

有外壳: 124.6*90.2*40mm (长宽高)

● 3.2 连接器接口

3.2.1 USB 接口

连接器规格

- 接口类型：USB 2.0 方口

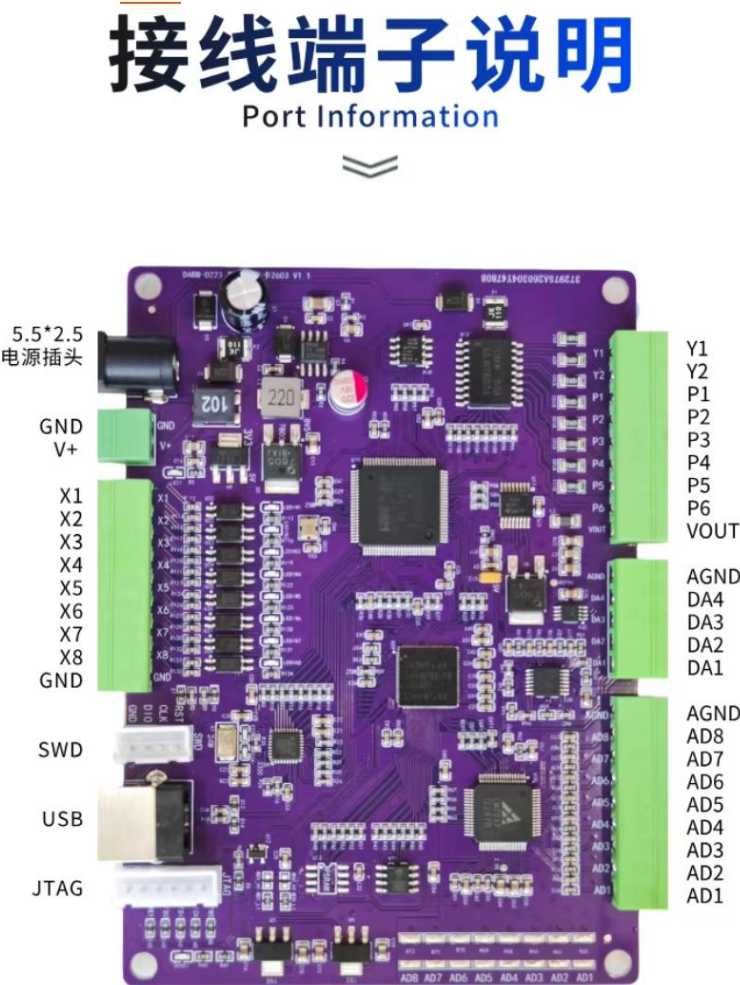
3.2.4 电源系统连接

3.81mm 接线端子电源输入

极性标识：V+：DC 12-24V 输入正极，GND：电源地

5.5*2.5 电源输入

3.3 端子定义及排序说明



端口标识	端口功能
GND	电源输入负极
V+	电源电输入正极
EV	对外供电正极（NPN输出上拉电源）
Y1~Y4、P1~P4	DO数字输出、PWM输出
SWD	烧录固件接口，引脚定义从外到内分别为GND、DIO、CLK、RST
JTAG	烧录固件接口，引脚定义从外到内分别为GND、REF、TDO、TDI、TCK、TMS
AD1~AD8、AGND	模拟输入AI通道1~8、参考地
DA1~DA4、AGND	模拟输出AO通道1~4、参考地
X1~X8、GND	数字输入DI通道1~8、参考地

第4章 上位机软件使用

采集卡开源资料存放在百度网盘中,使用前请先安装对应软件

000. 软件安装包

链接: https://pan.baidu.com/s/liHu4itkS9t520e4fX_iWBg?pwd=i72s 提取码: i72s

数据采集卡资料(型号: DABM-D223)

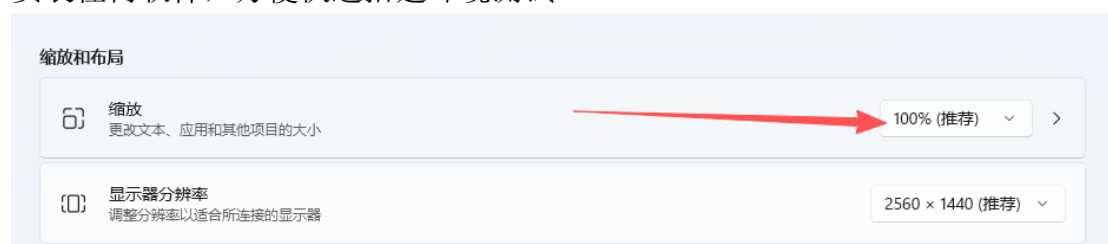
链接: https://pan.baidu.com/s/1I0DHYaZvRwDFkC6r_gX6iQ?pwd=jks3 提取码: jks3

单片机开发软件:CubeMx/KEIL MDK 5/rt-Thread 支持包

上位机开发软件:Visual Studio 2022 社区版/QT 5.14.2/labview2024



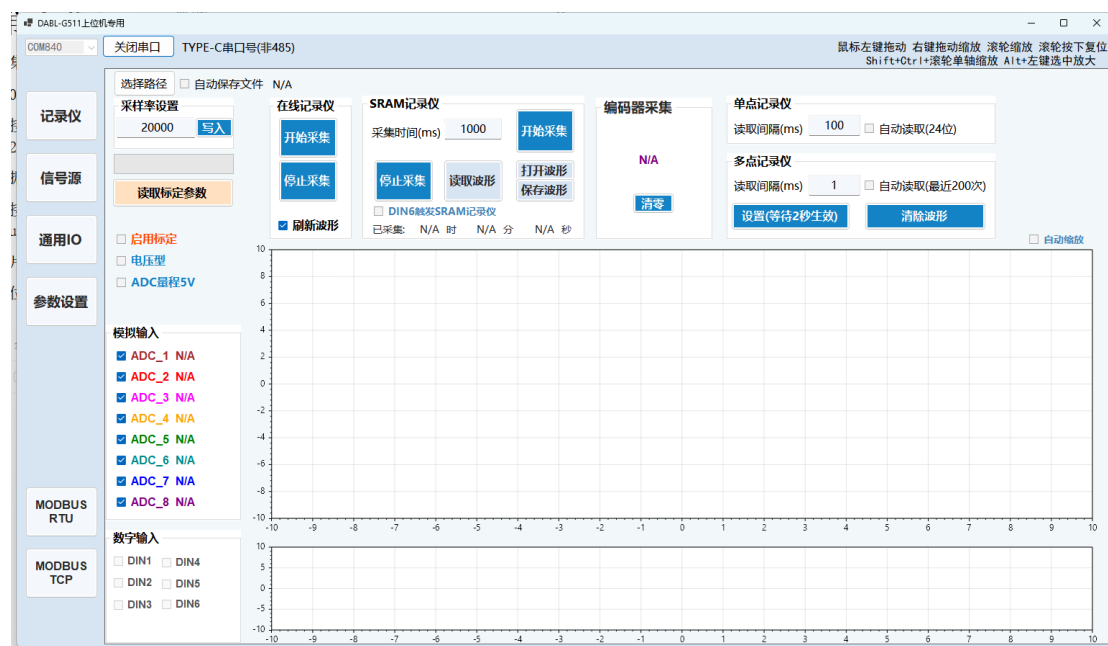
在上述截图中, 可以看到各类开源资料 其中硬件资料仅提供 PDF 格式原理图, LAYOUT 和 BOM 表等; C#压缩包为 Visual Studio 版本上位机, MDK 为单片机源码, qtTool 为 QT 版本上位机源码; zlTool 为免安装上位机, 可以直接使用无需安装任何软件, 方便快速搭建环境测试。



显示器推荐使用 1920*1080 以上分辨率，缩放改成 100%，不要对字体进行放大，否则会造成文字显示异常。

4.1 USB 连接

先将 USB 数据线连接板卡和 PC 端, 打开网盘下载的板卡资料, 找到 z1Tool 文件夹, 双击打开 z1Tool.exe 启动上位机。



在左上角端口号中选择设备的 COM 口，USB 虚拟串口波特率无需选择，选择好采集卡 USB 对应的串口之后点击下方”打开串口”，串口打开之后就不能再更改端口号和波特率的参数, 关闭串口后才可以，当串口颜色变浅，为不可选中状态时，则串口打开成功。



上位机界面的左侧分别有记录仪，信号源，通用 IO, 参数设置页面，点击后右侧子窗口惠跳转到相应页面。这 5 个按键对应的是采集卡 USB Type-B 接口，请勿使用 RS485 连接时打开此页面，否则数据不可读写。

USB 虚拟串口打开成功后，点击最左侧信号源，记录仪等按钮，上位机会读取相关页面参数，并跳转到此页面，就算现在仍然在当前页面，**点击当前页面跳转按钮仍然会读取页面参数，适合串口失效后重新开启使用。**

6 采集卡记录仪功能介绍

点击左侧记录仪按钮，进入示波器页面,USB 页面只有在 USB 方口线连接模式下才有效,接入采集卡，打开对应串口后,即可正常使用示波器功能。虚拟串口本身是 USB 协议,无需关注波特率。上位机启动时默认为记录仪页面，打开串口后，重新点击记录仪按钮可以更新本页面参数。



记录仪功能为本数据采集卡最核心的功能，目前做了 5 种记录仪功能，方便用户在不同的场景满足采集记录需求。

6.1 SRAM 记录仪

SRAM 记录仪使用板载的 SRAM 存储器进行数据记录。采集卡先对数据进行采集，采集完成后上传，可以多次读取，不容易因为误码导致数据丢失。支持外部 DIN 触发功能，可以在上位机配置是否使用 DIN8 进行触发采集，触发后会根据上位机预设的采样率与采集时间进行采集后上传，DIN 输入为 PNP 型，需要 NPN 型的数字接口对其接地边沿触发，板载 DIN8 输入灯亮的瞬间触发记录仪开始采集（如下图所示为 SRAM 记录仪相关控件）。最简单的 NPN 接口就是通过一个开关和数字输入的 GND 进行短接。

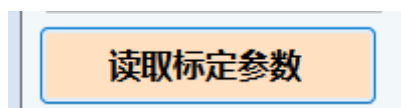


SRAM 记录仪的采样率为左图侧中设定，点击右侧开始采集按钮会自动把采样率、采集时间、通道使能状态等信息写入采集卡。

SRAM 采样率最高为 500K 不受 DAC 等其他外设影响。

电压型选项，点击勾选后，会切换成电流型采集，使用板载 240 欧电阻进行电流计算。

量程 5V，点击勾选后会切换到量程 10V，采集到的数据和显示的波形也会跟随变化。

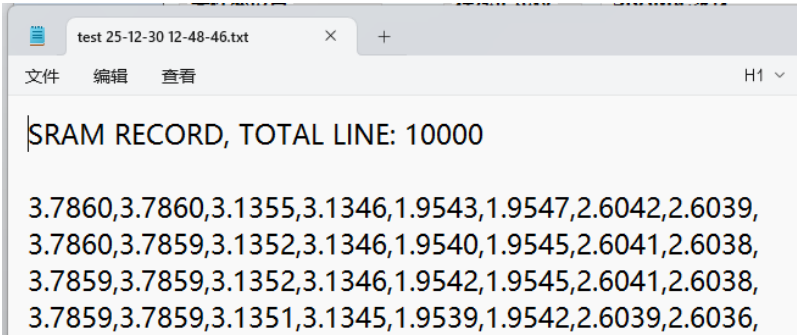


启用标定勾选前，需要先手动读取标定参数。勾选后，电流型勾选会被自动取消，按照电压型进行计算，计算细节参考设置页面标定功能，单位可以自定义，在设置页面选择切换，如下图所示。



采集过程中，已采集时间会按秒累加显示，提示记录仪在工作状态。

如果需要保存 SRAM 记录仪波形，需要读取波形后，点击开始采集下方的保存波形，对波形数据进行保存，保存后可以通过打开波形进行再次读取。



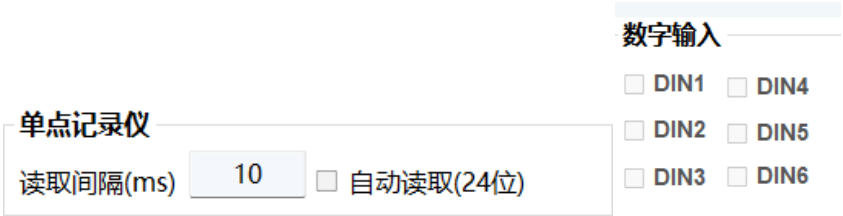
保存的数据第一行为记录仪类型，和数据总长度。第三行开始为具体数据，电压或者电流值，以逗号隔开，如果某通道未开启，则两个逗号中间没有任何数据，如下图所示，采集长度为 2 万条，通道 1, 3, 5, 7 开启有正常电压/电流读数，通道 2, 4, 6, 8 无数据，通道未开启。



SRAM 记录仪在未断电之前，数据会一直在存储器中不丢失，如果点击开始采集，则数据会被覆盖，需要确保数据已经完整存储再开始下一轮采集！

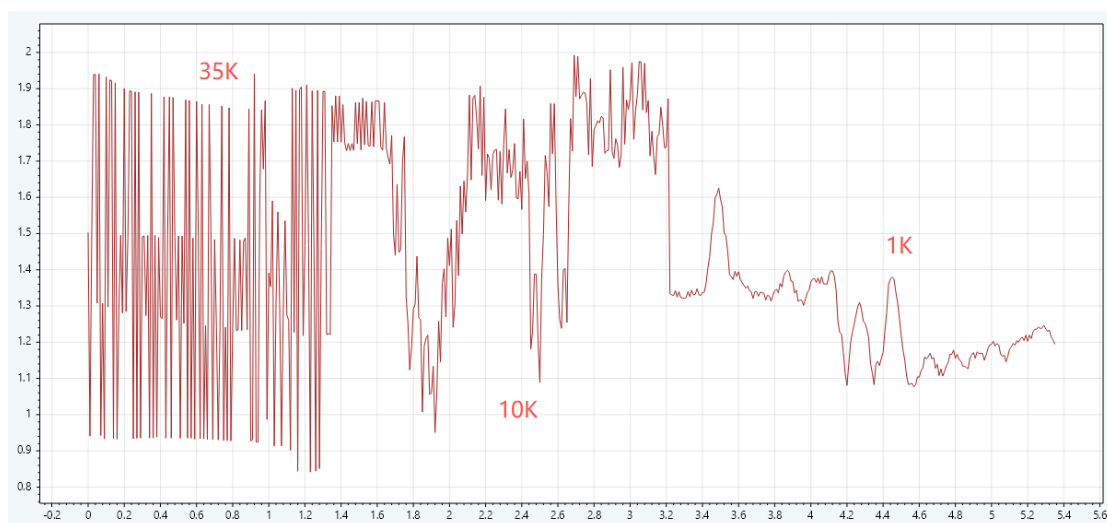
6.3 单点记录仪

根据上位机定时器，进行 24 位采集 一般不超过 100Hz。读取 ADC 的同时也会读取 DIN 状态，方便一条指令同时监测多个接口状态。采样率通过修改单点记录仪的读取间隔来调整，取值范围可以根据需求任意设置，受限于电脑 windows 响应速率，一般不会超过 100Hz（上位机数值为 10），此处最大为 32 位数字，可以写的很大，用来存储长时间大跨度的数据。



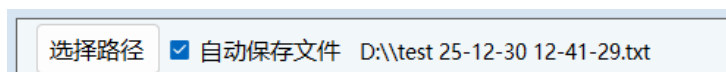
此处 ADC 是 16 位 SAR ADC，通过软件过采样的方式合成 24 位使用。软件累加 256 次，通过随机噪声平均值提取低于 SAR ADC 的微小信号，分辨率 24 位情况下，实际有效位数近似 20 位。如果要提高数据响应速率与信号带宽，可以适当调高 ADC 的采样率设置。采样率设置在此处有低通滤波器的效果，如下图所示，

在 100Hz 采集速率下，测工频干扰信号，35K 采样率下可以看到较多细节，10K 采样率下，信号特征明显减少，1K 采样率下，细节明显丢失，但是轮廓基本不变。

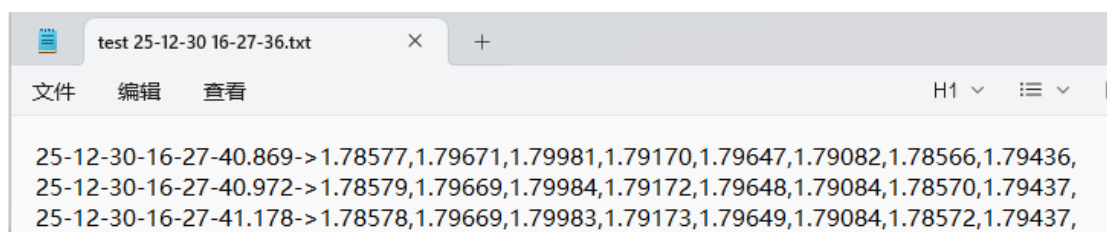


目前没有为写入采样率设置单独的写入指令，可以借用 SRAM 记录仪或在线记录仪等工具的开始采集功能用来写入采样率，并验证效果。

记录仪左上角可以选择自动存储路径，单点记录仪在实时显示波形过程中还能连续存储数据到本地电脑硬盘，如下图所示：



保存后的数据如下图所示，最左侧是 ASCII 形式日期，右侧分别是各个通道的电压、电流值，以逗号隔开。单点记录仪是 8 个通道一起存储，不受通道使能状态影响。

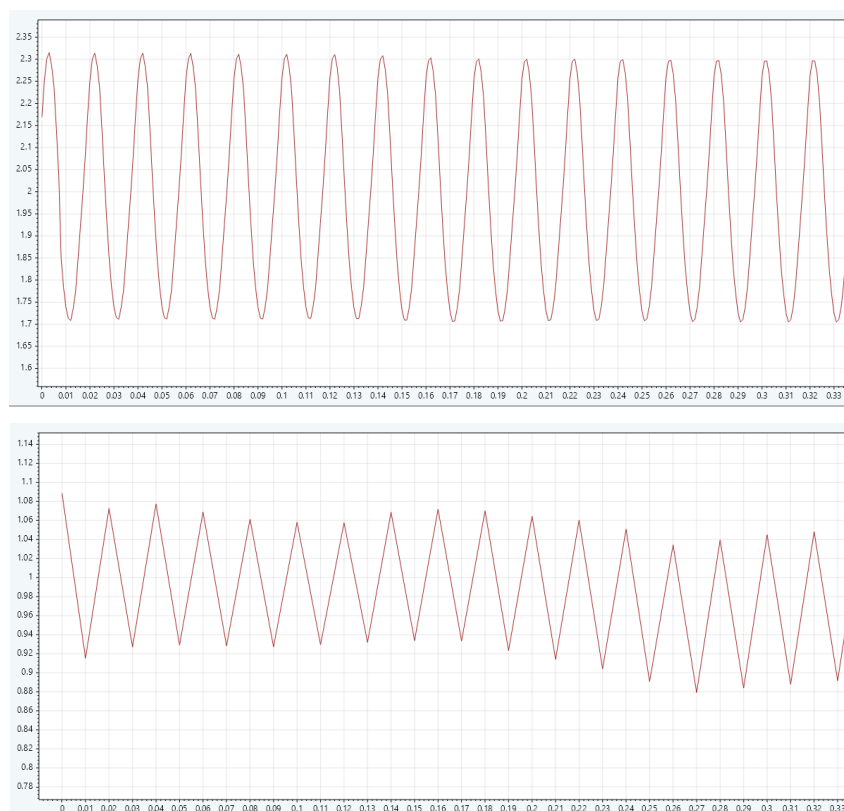


6.4 多点记录仪

100Hz-5K 采样率，16 位采集。多点记录仪的特点是可以 5K 采样速率在线观测数据并且同时缓存数据到电脑硬盘，在保留信号细节同时，实时还能观察到信号状态。

多点记录仪每次回读是 8 通道各 1000 次的历史数值，回读后上位机根据报文中的全局时间戳将数据裁剪出所需长度，再进行数据拼接，显示等。

如下图所示，测量 50Hz 工频干扰， 第一张图为 100Hz 采样率的波形， 第二张图为 1KHz 采样率的波形。对应的读取间隔分别是 10 毫秒和 1 毫秒。

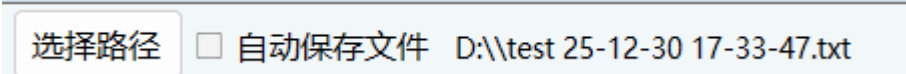


6.5 在线记录仪

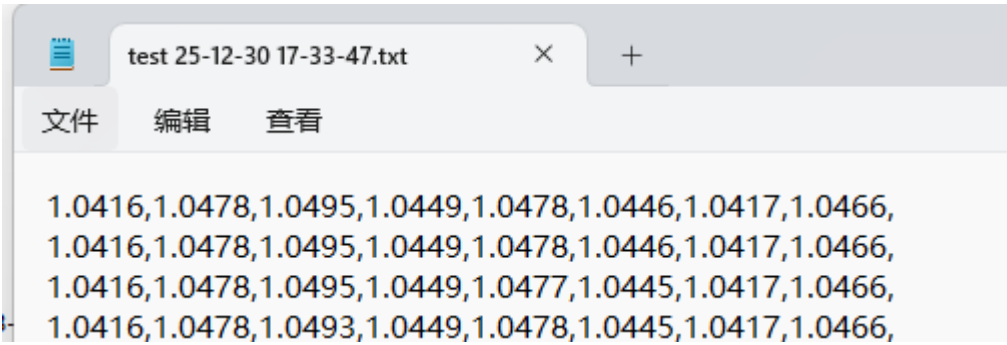
数据流形式缓存到硬盘，存储后再分析，容量不受采集卡限制 但是不适合在线观测，有丢包可能性，对电脑配置要求较高。采集过程中建议关闭刷新波形，可以大幅度节省电脑性能



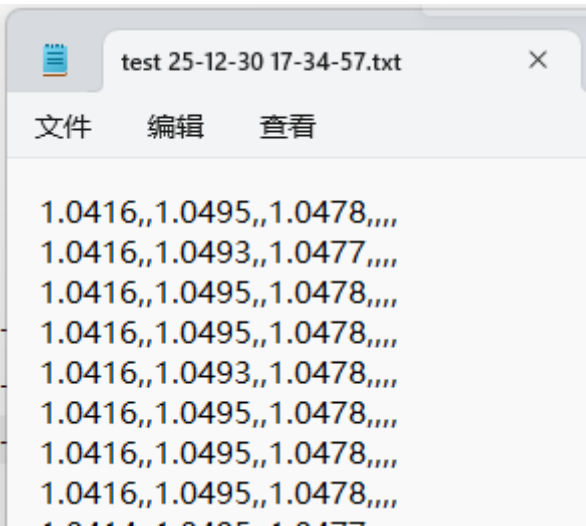
在线记录仪存储文件方式和单、多点记录仪相同，选择路径和自动保存文件同时存在，即可自动往硬盘中缓存数据。



采集的数据格式如下图所示，数据没有前缀，以逗号隔开。



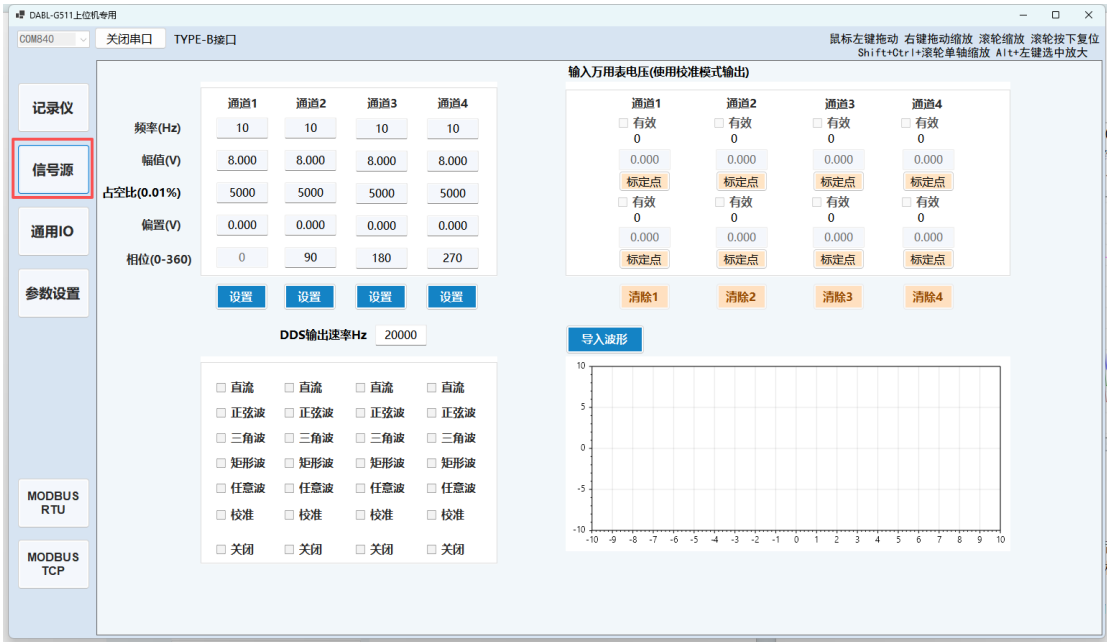
如果通道未开启，则两个逗号中间为空，如下图所示，通道 1, 3, 5 有采集数据回传，其余通道未开启。



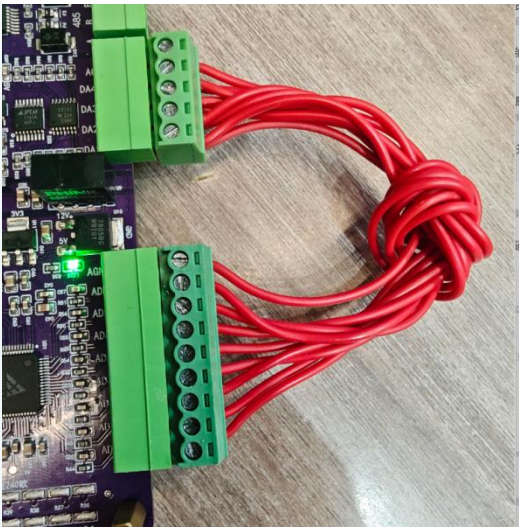
目前实测在线记录仪，USB 接口最大极限速率是 50K。测试电脑的处理器是 14 代 i9, 接口为百兆网。

7 采集卡信号源功能

点击左侧**信号源**按钮，进入**信号源**页面,USB 页面只有在 USB Type-C 模式下才有效,接入采集卡，打开对应串口后,即可正常使用**信号源**功能。虚拟串口本身是 USB 协议,无需关注波特率。重新点击**信号源**按钮可以更新本页面参数。



7.1 信号源直流功能



勾选信号源的直流功能后，发现参数页面只有幅值（电压值）可以设定。修改电压后，点击设置，即可修改 DAC 输出电压值，我们把 4 个通道的 DAC 输出分别接到采集卡的 ADC 输入引脚，以便观察 DAC 实际输出波形。

从下图中可以看出，4 个通道的电压都在 8V 左右，为直流信号。采集卡的 ADC 分辨率较高，用来观察 DAC 波形，可以看出很多细节。以 20K 采样率为例，DAC 误差大概在 5mV，纹波约 1mV，直接使用 DAC 信号源精度已经较高，并且还可以通过标定方式提高信号精度，会在后续 DAC 标定章节详细讲解。



7.2 正弦波功能

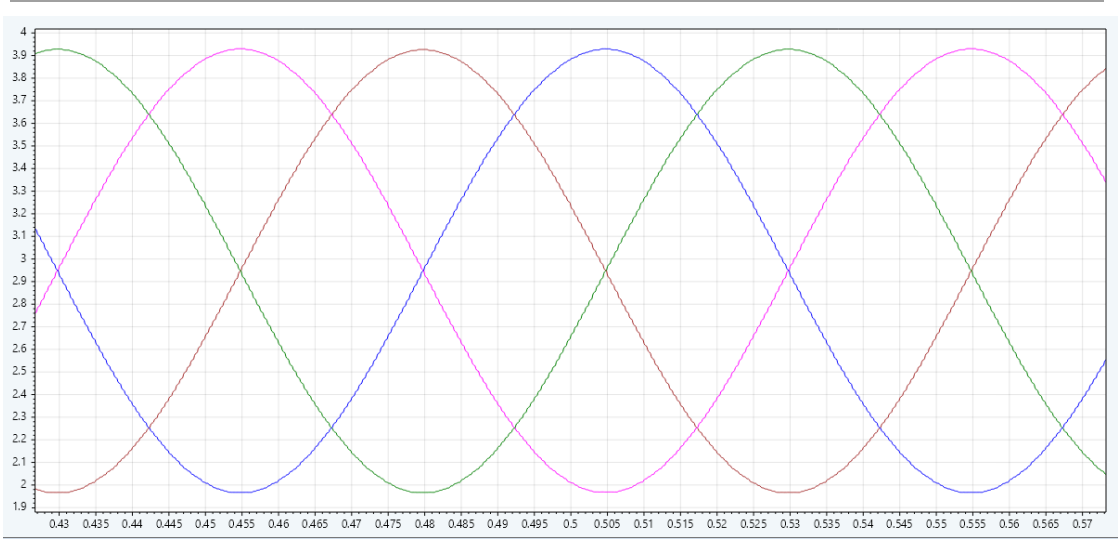
勾选信号源的正弦波功能后，发现参数页面频率、幅值、偏置、相位可以设定。其中通道 1 的相位为基准相位，数值强制为 0 无法修改。其余通道相位值为相对通道 1 来计算。

修改参数后，点击设置，即可修改 DAC 输出电压值，通道记录仪 ADC 观察 DAC 实际输出波形。

	通道1	通道2	通道3	通道4
频率(Hz)	10	10	10	10
幅值(V)	2.000	2.000	2.000	2.000
占空比(0.01%)	5000	5000	5000	5000
偏置(V)	2.000	2.000	2.000	2.000
相位(0-360)	0	90	180	270
	设置	设置	设置	设置

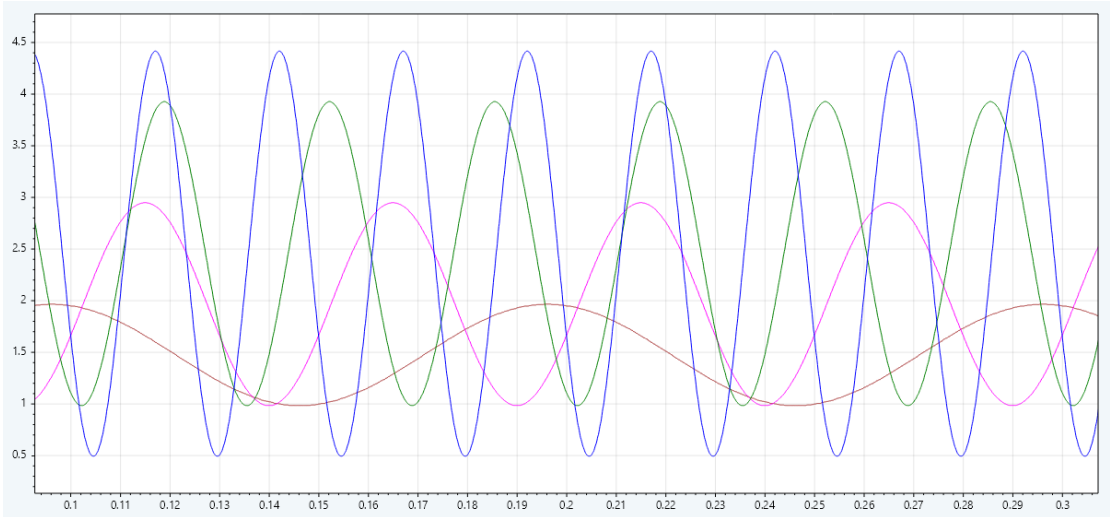
☐ 直流	☐ 直流	☐ 直流	☐ 直流
☒ 正弦波	☒ 正弦波	☒ 正弦波	☒ 正弦波
☐ 三角波	☐ 三角波	☐ 三角波	☐ 三角波
☐ 矩形波	☐ 矩形波	☐ 矩形波	☐ 矩形波
☐ 任意波	☐ 任意波	☐ 任意波	☐ 任意波
☐ 关闭	☐ 关闭	☐ 关闭	☐ 关闭
☐ 校准	☐ 校准	☐ 校准	☐ 校准

使用 10K 记录仪读取 DAC 波形，发现信号起始电压约为 2V，对应 DAC 的偏置电压，正弦波的峰峰值约为 2V，对应信号源的幅值，以桃红色波形为例，两个峰值的时间从 0.455 秒到 0.555 秒，大概为 0.1 秒，对应 10Hz，每个正弦波之间互相差值约 0.025 秒，对应 90 度相位差。



相位功能只有在同类型信号源，同频率下才会生效，如果信号源种类不同，或者频率不同，则信号之间相位值随机如下图所示：

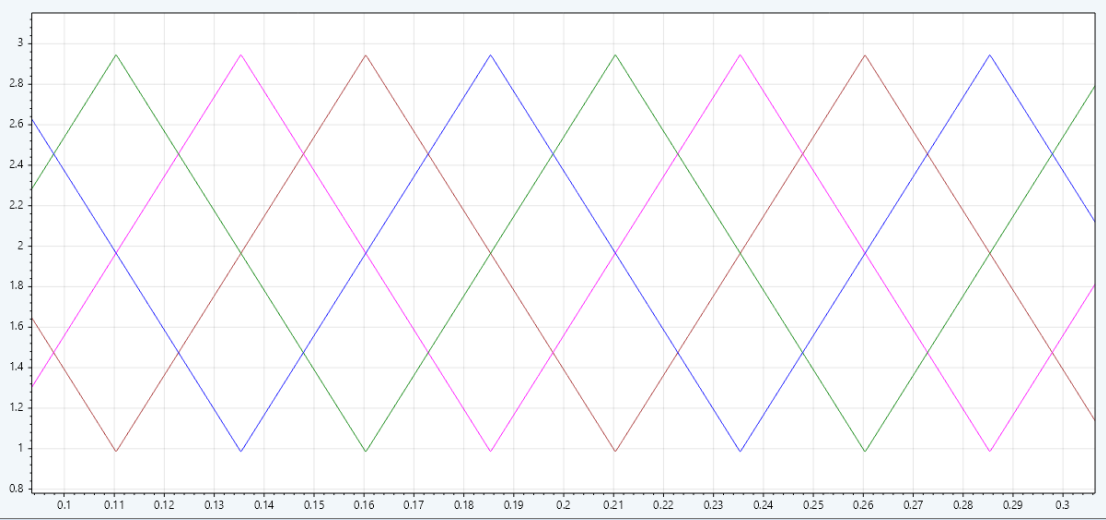
	通道1	通道2	通道3	通道4
频率(Hz)	10	20	30	40
幅值(V)	1.000	2.000	3.000	4.000
占空比(0.01%)	5000	5000	5000	5000
偏置(V)	1.000	1.000	1.000	0.500
相位(0-360)	0	90	180	270
	设置	设置	设置	设置



随便修改一些参数后，观察记录仪的 DAC 波形，对照设定值，可以发现频率、幅值、偏置功能在正常工作，相位功能已经失效。

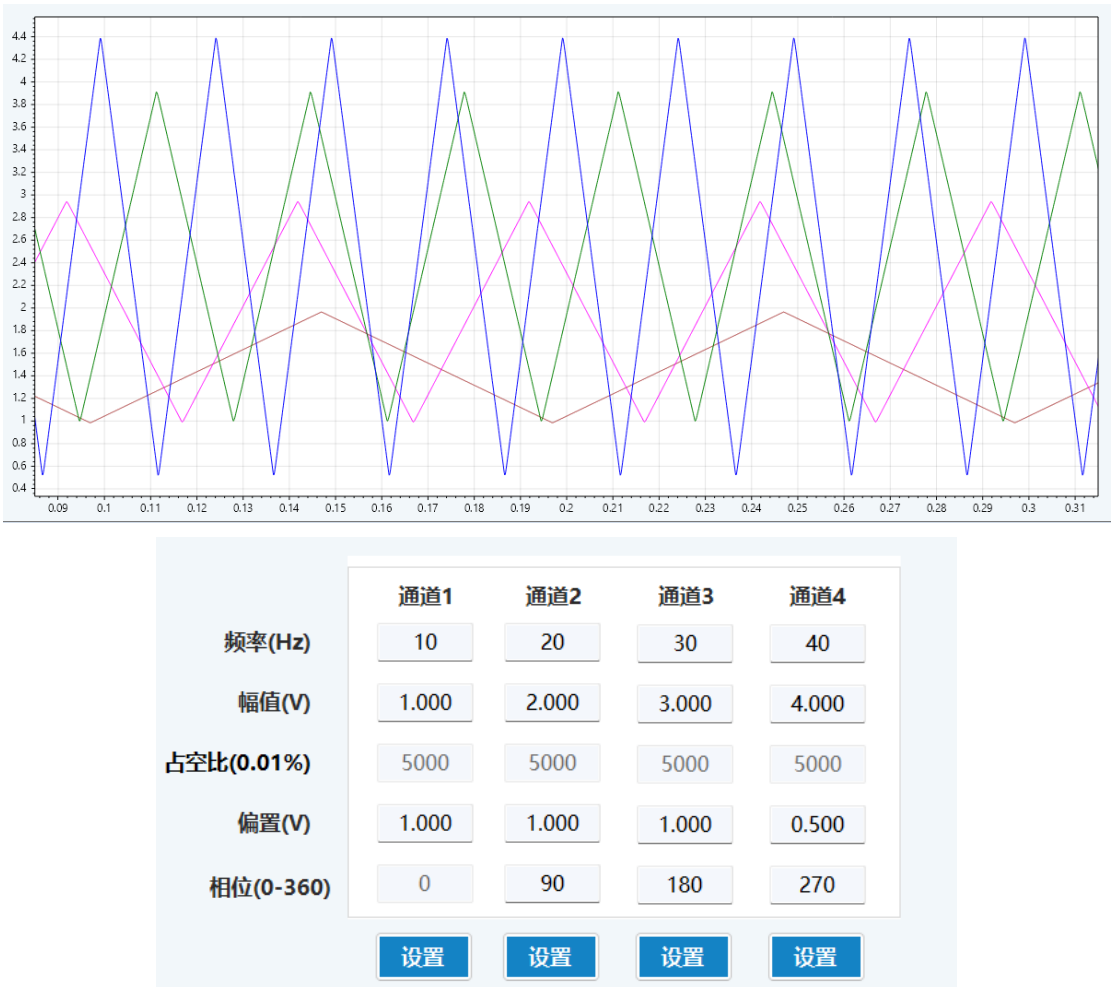
7.3 三角波功能

三角波功能和正弦波近似，也是有频率、幅值、偏置、相位可以设置。



	通道1	通道2	通道3	通道4
频率(Hz)	10	10	10	10
幅值(V)	2.000	2.000	2.000	2.000
占空比(0.01%)	5000	5000	5000	5000
偏置(V)	1.000	1.000	1.000	1.000
相位(0-360)	0	90	180	270
	设置	设置	设置	设置

☐ 直流	☐ 直流	☐ 直流	☐ 直流
☐ 正弦波	☐ 正弦波	☐ 正弦波	☐ 正弦波
☒ 三角波	☒ 三角波	☒ 三角波	☒ 三角波
☐ 矩形波	☐ 矩形波	☐ 矩形波	☐ 矩形波
☐ 任意波	☐ 任意波	☐ 任意波	☐ 任意波
☐ 关闭	☐ 关闭	☐ 关闭	☐ 关闭
☐ 校准	☐ 校准	☐ 校准	☐ 校准



此处就不再赘述，详细介绍参考正弦波部分，截取两部分不同参数的波形截图供参考。

7.2 矩形波功能

勾选信号源的矩形波功能后，发现参数页面频率、幅值、占空比、偏置、相位、可以设定。其中通道 1 的相位为基准相位，数值强制为 0 无法修改。其余通道相位值为相对通道 1 来计算。

修改参数后，点击设置，即可修改 DAC 输出电压值，通道记录仪 ADC 观察 DAC 实际输出波形。



频率(Hz)

幅值(V)

占空比(0.01%)

偏置(V)

相位(0-360)

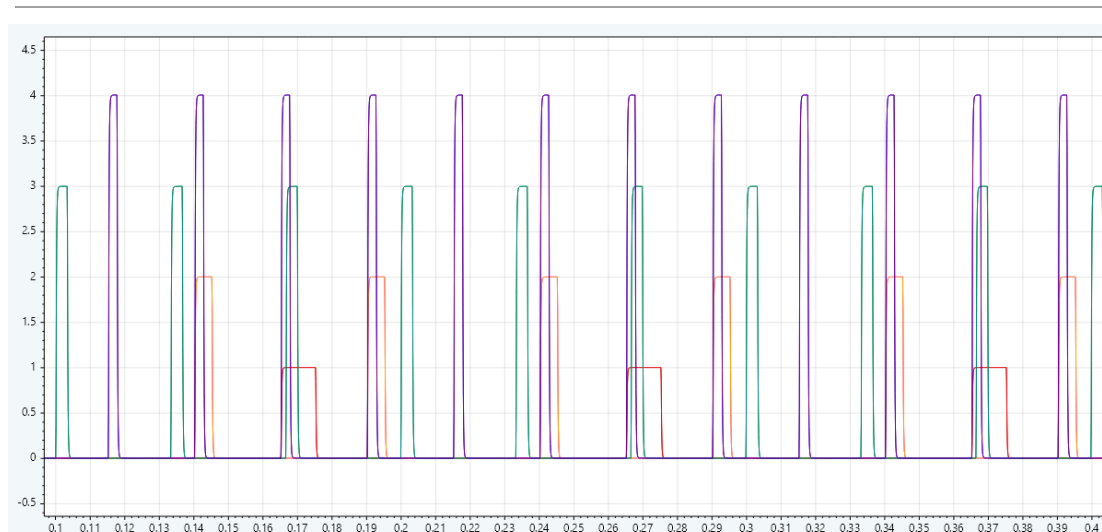
通道1	通道2	通道3	通道4
10	20	30	40
1.000	2.000	3.000	4.000
1000	1000	1000	1000
0.000	0.000	0.000	0.000
0	90	180	270

设置

设置

设置

设置



此处就不再赘述，详细介绍参考正弦波部分，截取两部分不同参数的波形截图供参考。

7.4 任意波功能

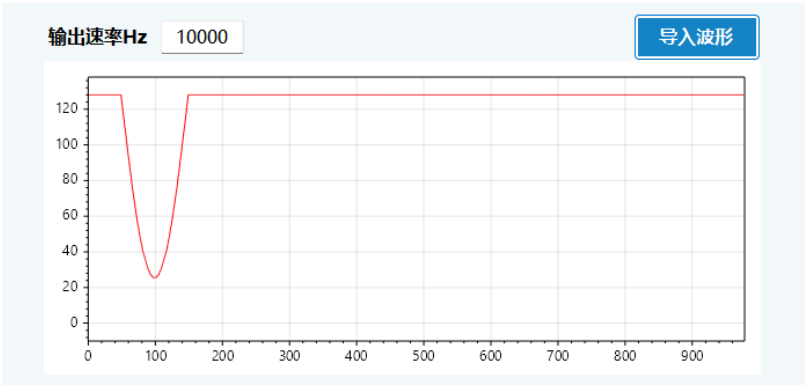
勾选信号源的任意波功能后，发现参数页面只有偏置和相位可以设定。其中通道 1 的相位为基准相位，数值强制为 0 无法修改。其余通道相位值为相对通道 1 来计算。

修改参数后，点击设置，即可修改 DAC 输出电压值，通道记录仪 ADC 观察 DAC 实际输出波形。

	通道1	通道2	通道3	通道4
频率(Hz)	10	10	10	10
幅值(V)	1.000	1.000	1.000	1.000
占空比(0.01%)	1000	1000	1000	1000
偏置(V)	1.000	1.000	1.000	1.000
相位(0-360)	0	90	180	270
	设置	设置	设置	设置

☐ 直流	☐ 直流	☐ 直流	☐ 直流
☐ 正弦波	☐ 正弦波	☐ 正弦波	☐ 正弦波
☐ 三角波	☐ 三角波	☐ 三角波	☐ 三角波
☐ 矩形波	☐ 矩形波	☐ 矩形波	☐ 矩形波
☒ 任意波	☒ 任意波	☒ 任意波	☒ 任意波
☐ 关闭	☐ 关闭	☐ 关闭	☐ 关闭
☐ 校准	☐ 校准	☐ 校准	☐ 校准

确保串口打开状态， 点击信号源页面右侧的导入波形，选择所需要的波形文件，双击文件或者点击打开即可烧录波形到采集卡，同时会将波形文件显示在上位机波形控件中， 如下图所示：



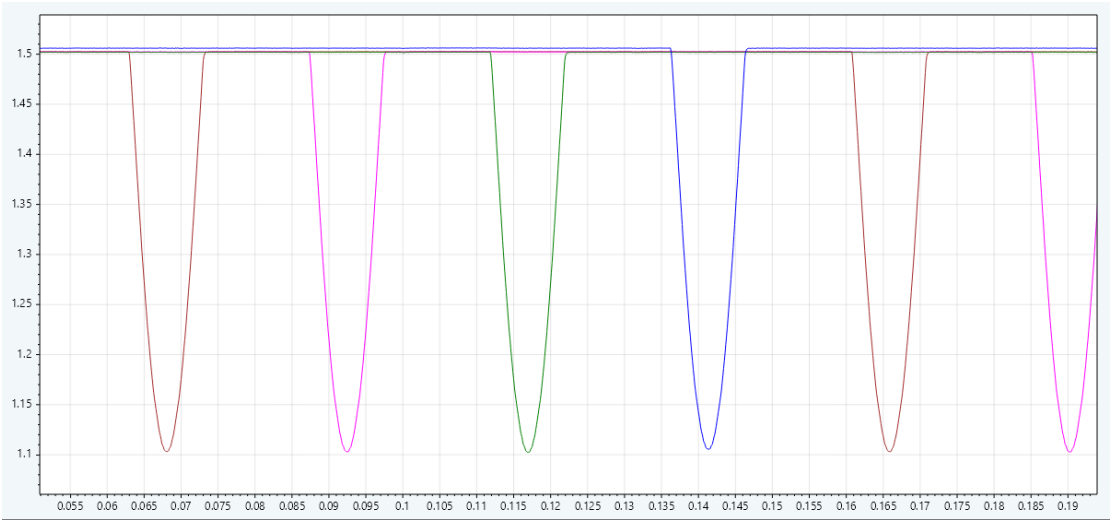
四个通道可以同时输出任意波， 并且相位和偏置可调。
任意波输出频率由查找表深度和输出速率（等同于 ADC 采样率）共同决定，频率等于输出速率除以查表深度。
任意波输出幅值由查找表数值决定，0 代表电压 0V， 1000 代表电压 5V，线性化分换算。

testData.txt

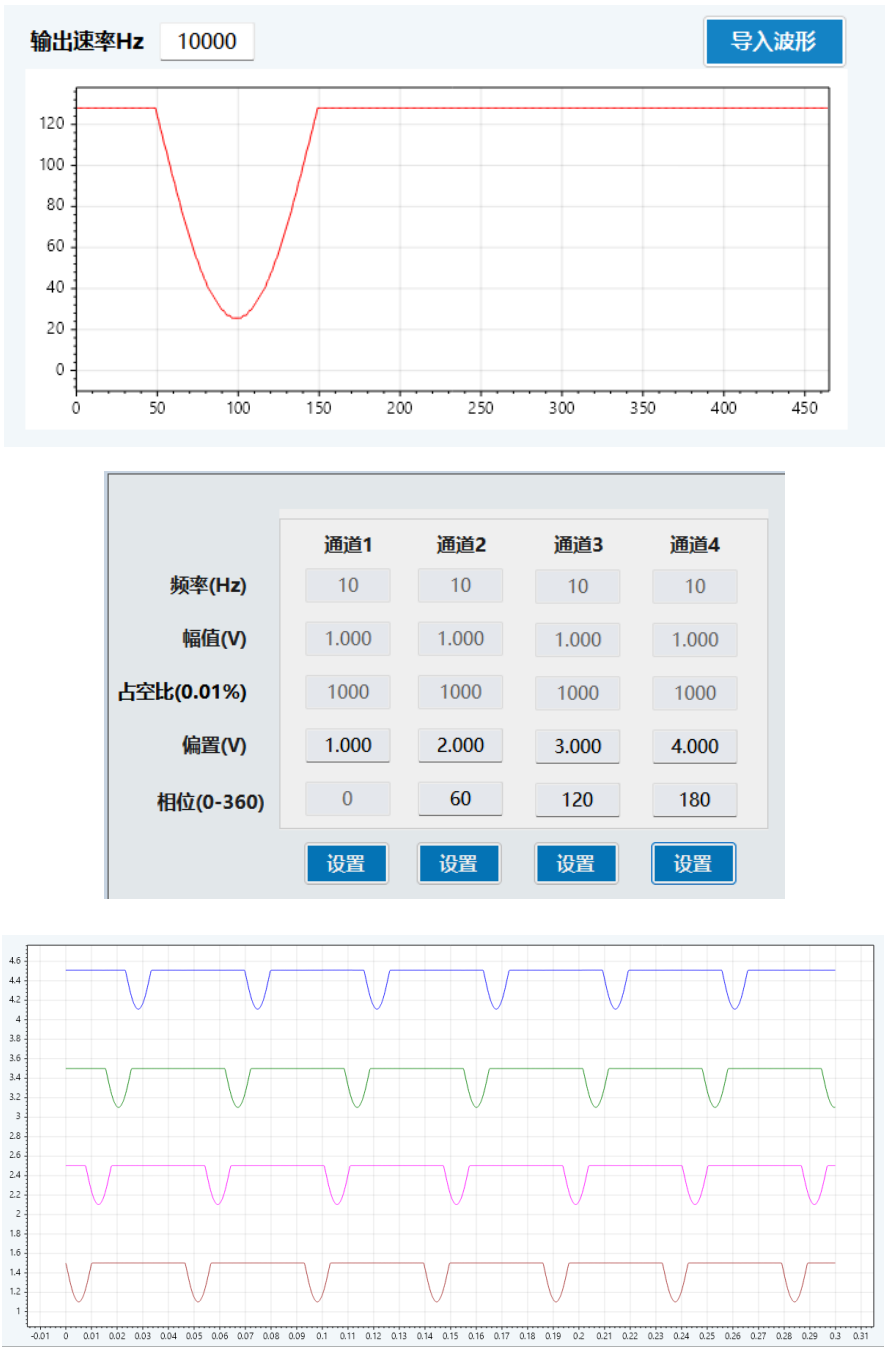
文件	编辑	查看
100 ,		
100 ,		
100 ,		
100 ,		
97 ,		
95 ,		
92 ,		
90 ,		
87 ,		
85 ,		
83 ,		
80 ,		
78 ,		
75 ,		
73 ,		
71 ,		
68 ,		
66 ,		
64 ,		
61 ,		
59 ,		
57 ,		
55 ,		
53 ,		
51 ,		
49 ,		
47 ,		
45 ,		
43 ,		
42 ,		
40 ,		

信号源查找表文件为 txt 格式，命名无要求，一行代表 1 个查找表数据，以逗号结尾，数值为 0-10000 的正整数，请勿在查找表文件中放入浮点数，字母，汉字等其他内容。查找表最大深度为 10000,超过 10000 的部分会被软件丢弃掉，不满 10000 的时候按照实际长度导入。

通过 ADC，回环测试采集到 DAC 波形如下图所示：



将波形文件深度删除到 500 左右， 波形不变， 可以看到波形宽度比例明显提升。修改偏置，相位，借助 SRAM 记录仪采集波形，效果如下图所示：



7.5 信号源标定功能

信号源支持标定功能,本采集卡的DAC是PWM加二级低通滤波器实现的DAC,使用的电压基准为 TI-REF5050,精度本身已经较高,如果对输出精度有更高要求,使用前可以通过标定提升输出电压精度。

标定方法很简单,将采集卡的模式切换到标定模式,任意设置输出两个点的电压值,通过外部仪表测量实际电压,写入采集卡即可。采集卡会根据外部反馈的电压自动调整输出参数,尽量匹配预设值。此处以 ADC 和 DAC 回环测试为例,

可以用来辅助 DAC 的标定。

☐ 直流	☐ 直流	☐ 直流	☐ 直流
☐ 正弦波	☐ 正弦波	☐ 正弦波	☐ 正弦波
☐ 三角波	☐ 三角波	☐ 三角波	☐ 三角波
☐ 矩形波	☐ 矩形波	☐ 矩形波	☐ 矩形波
☐ 任意波	☐ 任意波	☐ 任意波	☐ 任意波
☒ 校准	☒ 校准	☒ 校准	☒ 校准
☐ 关闭	☐ 关闭	☐ 关闭	☐ 关闭

将信号源的输出模式切换到校准，在切换之前，校准参数为灰色不可选中。切换之后才可以设置参数。校准模式的信号输出和直流模式相同，输出直流电压，通过反馈来进行自调节。

Shift+Ctrl+滚轮单轴缩放 Alt+左

输入万用表电压(使用校准模式输出)

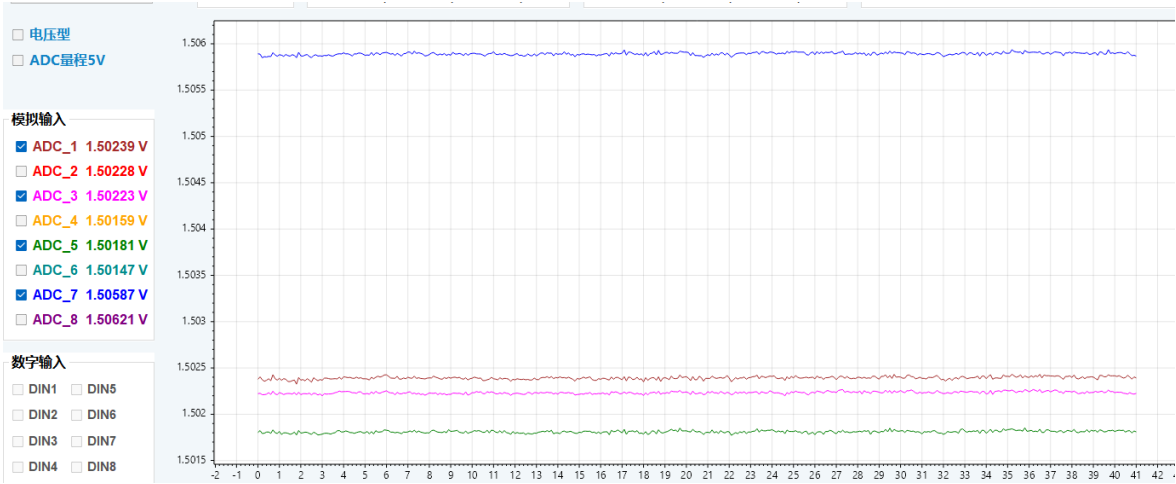
通道1 ☐ 有效 0 0.000 标定点 ☐ 有效 0 0.000 标定点 清除1	通道2 ☐ 有效 0 0.000 标定点 ☐ 有效 0 0.000 标定点 清除2	通道3 ☐ 有效 0 0.000 标定点 ☐ 有效 0 0.000 标定点 清除3	通道4 ☐ 有效 0 0.000 标定点 ☐ 有效 0 0.000 标定点 清除4
---	---	---	---

以标定 2 个点，1.5V 和 3.5V 举例。标定常用电压值为 30%-70%，可以根据实际需求进行调整，推荐范围为使用量程的 30%和 70%，可以让标定线性度尽量提升。

设置信号源输出电压为 1.5V，同时将信号源输出接入到 ADC 采集，通过 ADC 的单点记录仪（24 位模式）进行电压测量。

	通道1	通道2	通道3	通道4
频率(Hz)	10	10	10	10
幅值(V)	1.500	1.500	1.500	1.500
占空比(0.01%)	1000	1000	1000	1000
偏置(V)	1.000	2.000	3.000	4.000
相位(0-360)	0	60	120	180
	设置	设置	设置	设置

实际测量电压值如下图所示，将输出电压填入信号源，并点击写入。



将 ADC 测量的电压值按顺序填入到 DAC 标定框 A 点。

输入万用表电压(使用校准模式输出)

通道1	通道2	通道3	通道4
☒ 有效 300	☒ 有效 300	☒ 有效 300	☒ 有效 300
1.502	1.502	1.501	1.505
标定点A	标定点A	标定点A	标定点A
☐ 有效 0	☐ 有效 0	☐ 有效 0	☐ 有效 0
0.000	0.000	0.000	0.000
标定点B	标定点B	标定点B	标定点B
清除1	清除2	清除3	清除4

点击标定点 A 后，采集卡会自动回传当前 DAC 电压值对应的 DAC 数字量，并

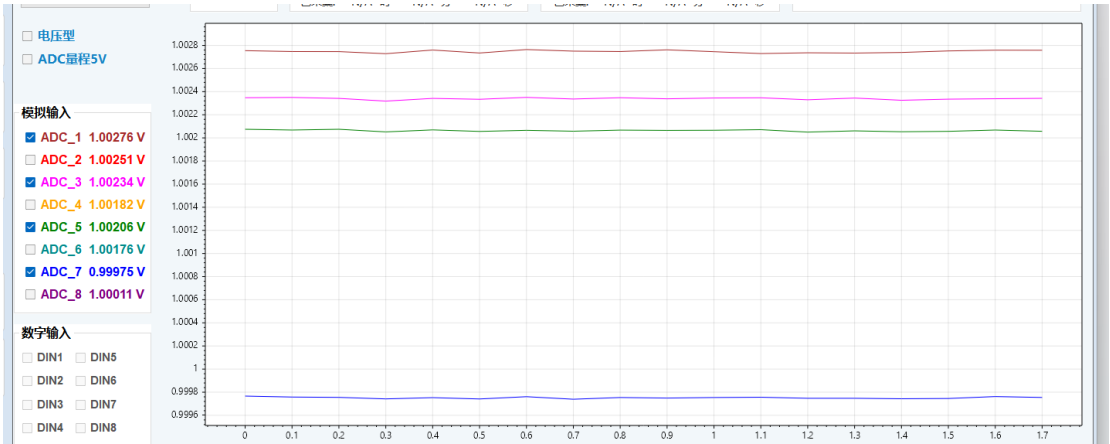
将有效位选中。

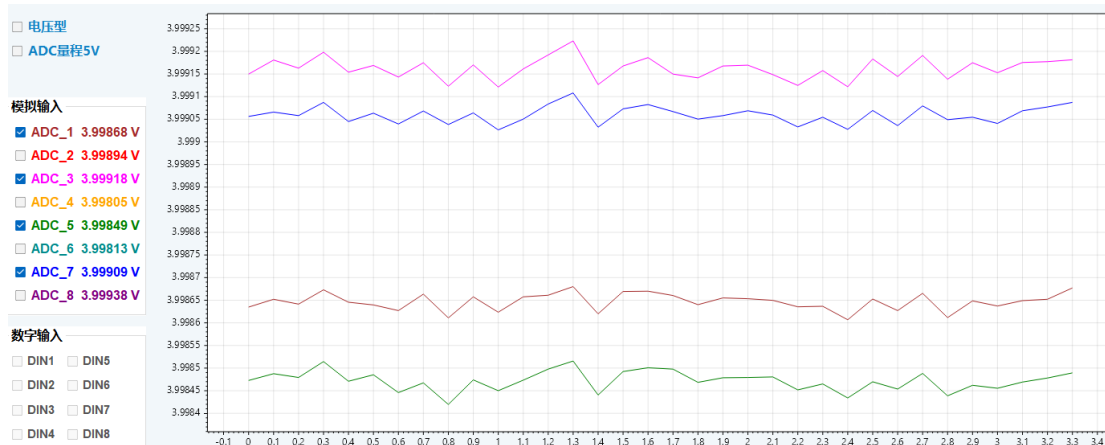
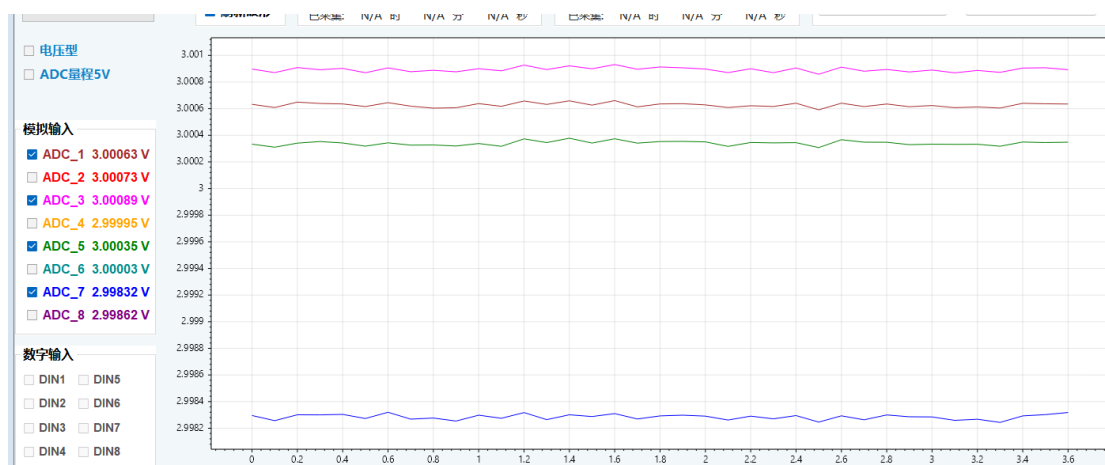
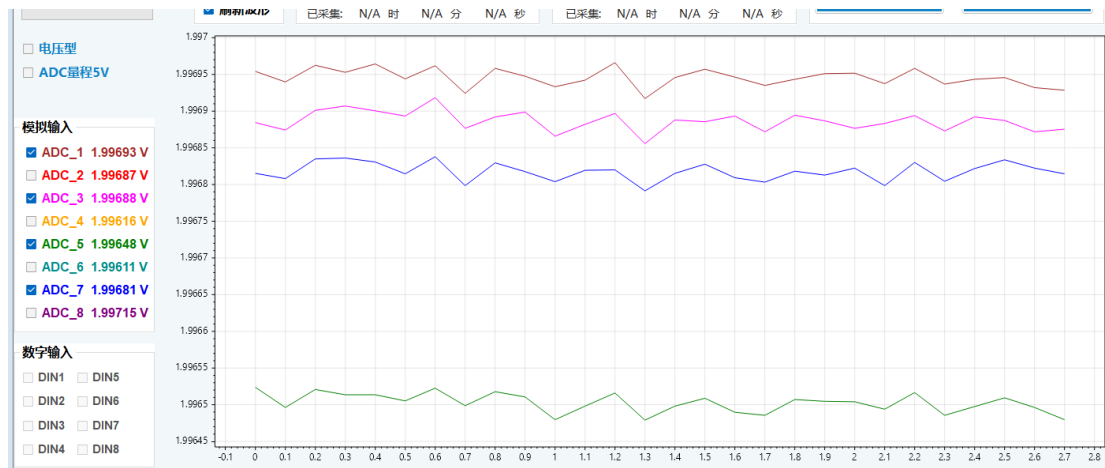
同样的方式将输出电压设置为 3.5V，并将 ADC 采集的电压填入到标定点 B，如下图所示：



当标定 A 点和标定 B 点同时为有效时，DAC 输出才按照标定模式输出，只要有任何 1 个点未有效，DAC 按照理论值输出，标定参数不参与计算。

下面四张图是标定完成后，实测输出 1V、2V、3V、4V 的回环效果(此图参考 DABL-7606)。





DAC 标定电压输出值是一方面，更大的作用是可以按照传感器实际输出值来进行标定，比如 2V 代表 10mA 的时候， 只要用类似的方式将外部传感器值标入采集卡，即可完成传感器输出的标定。

对于已经标定好的采集卡，如果想清除标定参数，点击清除即可。

输入万用表电压(使用校准模式输出)

通道1 ☐ 有效 0 0.000 标定点A ☐ 有效 0 0.000 标定点B	通道2 ☐ 有效 0 0.000 标定点A ☐ 有效 0 0.000 标定点B	通道3 ☐ 有效 0 0.000 标定点A ☐ 有效 0 0.000 标定点B	通道4 ☐ 有效 0 0.000 标定点A ☐ 有效 0 0.000 标定点B
清除1	清除2	清除3	清除4

清除后，所有参数恢复到默认状态。

8 采集卡 IO 功能

点击左侧通用 IO 按钮，进入通用 IO 页面,USB 页面只有在 USB Type-C 模式下才有效,接入采集卡，打开对应串口后,即可正常使用通用 IO 功能。虚拟串口本身是 USB 协议,无需关注波特率。重新点击通用 IO 按钮可以更新本页面参数。

DABL-G511上位机专用

COM540 关闭串口 TYPE-B接口

鼠标左键拖动 右键拖动缩放 滚轮缩放 滚轮按下复位 Shift+Ctrl+滚轮单轴缩放 Alt+左键选中放大

记录仪

信号源

通用IO

参数设置

MODBUS RTU

MODBUS TCP

☐ 自动回读

数字输入DIN

☐ DIN1

☐ DIN2

☐ DIN3

☐ DIN4

☐ DIN5

☐ DIN6

数字输出DOUT

☐ DO1

编码器采集

0

清零

脉冲输出

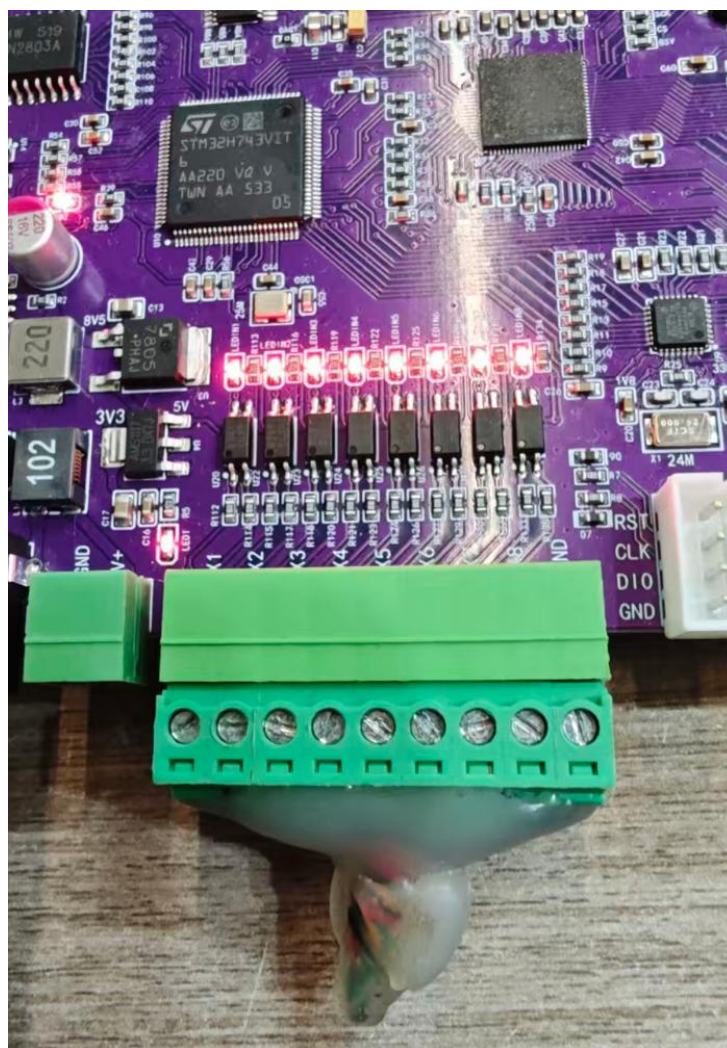
	频率(Hz)	占空比(0.1%)	脉冲数	加减速率	
☐ PWM使能1	1000	100.0	0	1	写入1
☐ PWM使能2	1000	100.0	0	1	写入2
☐ PWM使能3	1000	100.0	0	1	写入3
☐ PWM使能4	1000	100.0	0	1	写入4
☐ PWM使能5	1000	100.0	0	1	写入5
☐ PWM使能6	1000	100.0	0	1	写入6
☐ PWM使能7	1000	100.0	0	1	写入7

点击勾选自动回读，可以持续刷新自动回读本页面参数，适合对数字输入进行监控，或者 PWM 脉冲输出完成后的状态监控使用。

8.1 采集卡数字输入 DIN 功能

采集卡数字输入内部接线为 PNP 型，外部触发需要使用 NPN 接口，最简单的 NPN 触发方式，是将 DIN 的引脚和 GND 短接即可，可以用普通开关，脚踏开关触发，也可以用 NPN 型输入如 NMOS，达林顿管等接入 DIN 引脚。

DIN 触发后，PCB 板载的 LED 会亮起，如下图所示，DIN 接入短路帽，9 个引脚全部短接，可以同时触发所有 DIN 引脚。



数字输入DIN

☒ 自动回读

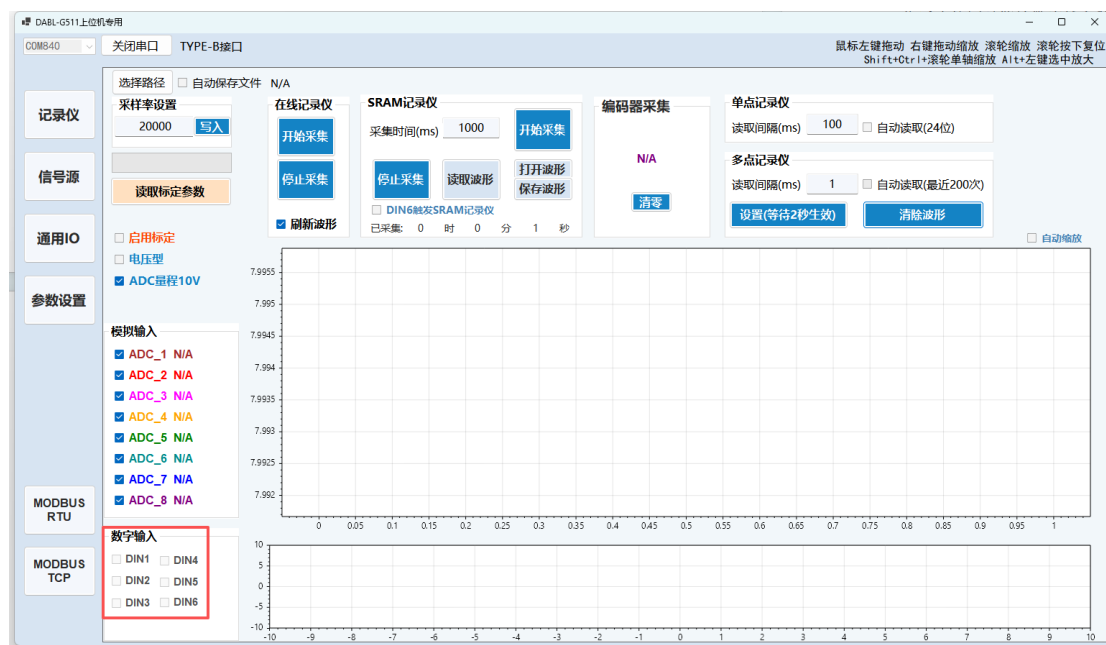
☒ DIN1 ☒ DIN5

☒ DIN2 ☒ DIN6

☒ DIN3 ☒ DIN7

☒ DIN4 ☒ DIN8

为了方便同时监控数字输入和模拟输入，记录仪页面的单点记录仪和多点记录仪，也会同时上传数字量状态，在记录仪页面左下角可以看到，如下图红框中所示，为 8 通道 DIN 实时状态。



8.2 采集卡数字输出 DOUT 功能

采集卡数字输出内部为 NPN 型，外部需要上拉负载才可以正常工作。最简单的驱动方式，是在采集卡的 VOUT 引脚和对应的 DO 引脚之间直接放置一个负载，比如电阻，继电器等。DO 驱动能力最大为 100mA，为了维持稳定开关效果，最小负载电流建议在 0.1mA 以上。

以外接继电器为例，DOUT 触发后，外部继电器会动作，可以听到继电器动作的声音，也会看到继电器的灯状态发生变化。



如上图所示, 为 D0 触发输出时, 继电器的状态。

8.3 采集卡数字输出 PWM 功能

采集卡 PWM 输出内部为 NPN 型, 外部需要上拉负载才可以正常工作。PWM 驱动能力最大为 100mA, 为了维持稳定开关效果, 最小负载电流建议在 0.1mA 以上。

PWM 可以驱动步进或者伺服电机驱动器, 用来发送脉冲控制电机

PWM 脉冲的频率范围为 1Hz-5KHz, 分辨率为 1Hz。

PWM 脉冲的占空比范围是 0%-100%, 分辨率为 0.1%。

PWM 的脉冲数设置范围为 0-65535, 为无符号 16 位整型, 如果脉冲数设置为 0 时, 脉冲不参与计数, 会持续发送脉冲, 直到手动关闭, 并且此时加减速斜率功能无效, 脉冲直接按照最大频率输出。

PWM 的加减速率为步进电机, 每 0.1 秒, 脉冲频率的增加值。比如设置为 10, 则每 0.1 秒, PWM 的频率增加 10。

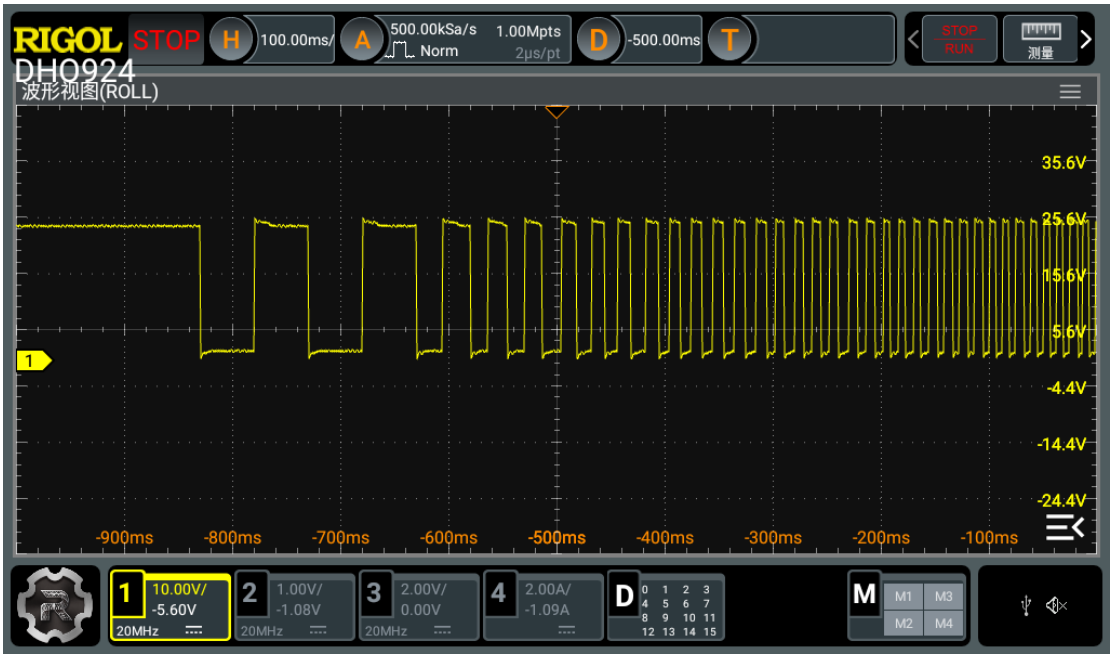
脉冲输出

	频率(Hz)	占空比(0.1%)	脉冲数	加减速率	
☐ PWM使能1	1000	100.0	0	1	写入1
☐ PWM使能2	1000	100.0	0	1	写入2
☐ PWM使能3	1000	100.0	0	1	写入3
☐ PWM使能4	1000	100.0	0	1	写入4
☐ PWM使能5	1000	100.0	0	1	写入5
☐ PWM使能6	1000	100.0	0	1	写入6
☐ PWM使能7	1000	100.0	0	1	写入7

PWM 的频率下限为 10Hz,防止电机运动到最后时刻速率太慢,运动时间过长。

PWM 在停止状态, TIM 仍在工作,频率约为 8.4Hz,这个是在尽量不占用中断资源,以及快速响应上位机指令来设计的参数,对运动指令的响应延时也为 8.4Hz(约±60 毫秒)。

PWM 最左侧为输出使能,勾选后控制脉冲开始发送(电机开始运动),取消勾选后,脉冲发送停止(电机停止运动)。,设置脉冲数后,脉冲数量发送完成,采集卡会自动上传一条运动完成的指令到上位机来取消 PWM 使能的勾选状态,用户也可以勾选本页面自动回读,来刷新运动状态。





9 设置页面

点击左侧参数设置按钮，进入参数设置页面，USB 页面只有在 USB Type-C 模式下才有效，接入采集卡，打开对应串口后，即可正常使用参数设置功能。虚拟串口本身是 USB 协议，无需关注波特率。重新点击参数设置按钮可以更新本页面参数。



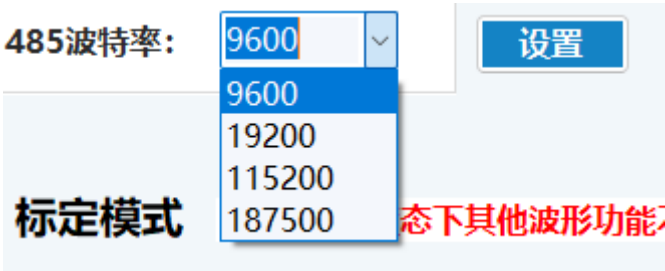
设置页面功能主要分 2 部分，上半部分为采集卡相关的参数设置；下半部分为采集卡 ADC 部分的标定功能，标定功能参数较多，所以放在此页。

9.1 采集卡参数设置

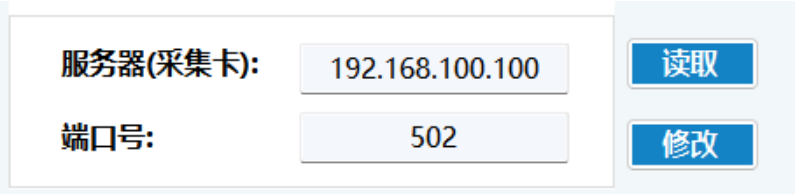
最上面一行是 MODBUS 参数设置，以及 flash 和 SRAM 记录仪回读过程中每一帧报文的发送延时。

MODBUS RTU 为 RS485 接口，其地址和波特率通过左上角参数设置。地址范围为 1-255，其中 0 为预留，请勿使用。波特率不支持自定义，预设了一部分常见波特率在采集卡中，如果需要其他波特率可以联系官方技术支持进行升级。

常见波特率如下图所示：



MODBUS TCP 的服务器 IP 地址和端口号通过最上侧中间栏参数设置，MODBUS TCP 的网段和上位机以及交换机的网段必须在同一个网段，也就是如下图所示，192.168.100.xxx，前三个数字和 MODBUS TCP 服务器以及交换机必须相同。MODBUS TCP 端口号常见为 502，也可以自行修改。



记录仪上传延时通过最右上侧参数设置。有些上位机电脑的处理性能较弱，CPU 主频比较低时，如果采集卡全速率上传，上位机软件的数据处理压力很大，在电脑性能不够的时候，会产生粘包，丢包等现象，导致上位机卡死，数据丢失等，此时可以酌情修改此数值，范围为 0-255, 0 代表全速率上传，255 代表每发送一个扇区数据（一般是 4000 多个字节），等待上位机处理的延时时间，数值越大，上传速率越慢，对电脑端上位机的压力越小，但是通信传输的时间也会越长，需要酌情修改。

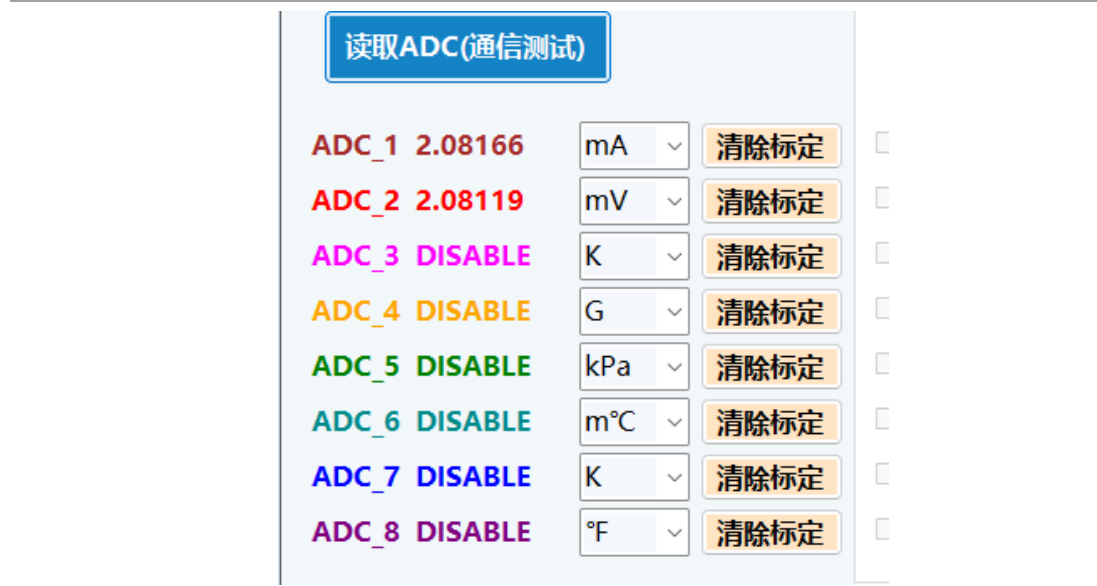


9.2 采集卡 ADC 校准功能

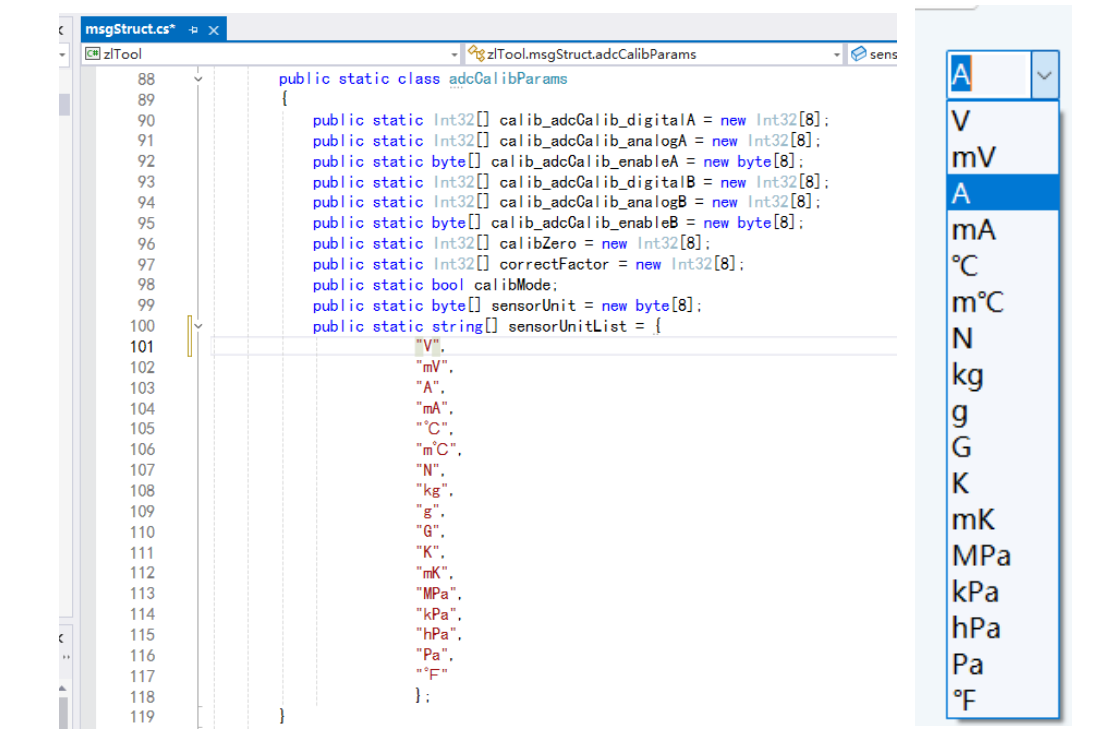
采集卡标定功能如下图所示，从左到右分别是读取当前 ADC 标定前电压值，标定后数值的单位，两点标定，修正系数和归零值。在修改标定参数前，必须先在此页勾选到标定模式选项才可以对标定参数进行修改。



勾选标定模式后，可以点击读取 ADC（通信测试）按钮，尝试读取 ADC 值。此操作的目的是，防止采集卡固件版本异常，或者串口未打开，标定模式未成功开启等原因造成的采集卡并未实际进入标定模式，如果点击后，ADC 的数值成功更新，则说明校准模式已经成功进入，如下图所示：



校准模式下，可以直接将采集卡显示的数值计算成外部传感器的其他物理量，且物理量的单位，每个通道都可以任意自定义。采集卡预设了一些单位在下拉框中，如果需要增加新的单位，可以在采集卡的 C# 上位机中，msgStruct.cs 文件的数组中直接增加单位即可，如下图所示：



在维持通信状态下，下拉框选中后即直接写入采集卡，无需再另外点击设置。单位设置成功后，回到记录仪页面，如下图所示，可以看到，记录仪读取的单位也已经跟着产生变化：

ADC_1 N/A

ADC_2 N/A

ADC_3 N/A

ADC_4 N/A

ADC_5 N/A

ADC_6 N/A

ADC_7 N/A

ADC_8 N/A

mA

mV

K

G

kPa

°F

K

°F

模拟输入

☒ ADC_1 0.05485 mA

☒ ADC_2 0.05485 mV

☒ ADC_3 0.05397 K

☒ ADC_4 0.05442 G

☒ ADC_5 0.05299 kPa

☒ ADC_6 0.05306 m°C

☒ ADC_7 0.05467 K

☒ ADC_8 0.05475 °F

再往右边是 A/B 两点标定功能，一般是根据满量程的 30%和 70%两点附近进行标定，然后线性化分割整个量程，可以将量程线性度最大化。

标定点A

状态	数字量	模拟量
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000

标定点B

状态	数字量	模拟量
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000
☐ 有效	0	0.00000

两点标定是假设整个量程都是线性的，数字量和实际模拟量关系为简单的一元一次方程 $Y=MX+N$ ，其中 Y 为计算的模拟量，X 为输入的 ADC 数字量。为了求得系数 M 和 N，至少需要两组参，所以通过 A 和 B 两组标定点，即可求出。

可以在采集卡的 C# 上位机中，msgStruct.cs 文件中查看具体计算过程，如图所示：

122
123
124
125
126
127

3 个引用

```
public static double calc_adcCalibData(double data, double d1, double a1, double d2, double a2)
{
    double result = (((data - d1) * (a2 - a1) / (d2 - d1)) + a1);
    return result;
}
```

其中，D1 为 A 点标定的数字量，D2 为 B 点标定的数字量，A1 为 A 点标定的模拟量，A2 为 B 点标定的模拟量，data 为当前实测的数字量，需要求出当前对应的模拟量 result。

标定过程，需要外部有高精度的仪器仪表，可以精确测量当前提供给采集卡的物理量对应的值，填入标定点的模拟量栏中，点击标定点即可。点击标定后，上位机会把用户输入的模拟量传输到采集卡中，采集卡会实时读取当前 ADC 通道对应的数字量，一起存入到存储器中，并将当前标定对应的点 A 或者点 B 的状态标记为有效。

有效无法手动修改，是软件自己处理的，为只读参数。

标定点A

状态

数字量

模拟量

☒ 有效

10042332

2.00000

标定点

标定点B

状态

数字量

模拟量

☒ 有效

11688477

4.00000

标定点

如上图所示，采集卡对输入的电压分别进行了 2V 和 4V 标定，数字量的值分别为 2V 时对应的 ADC 电压，以及 4V 时对应的 ADC 电压，都为 24 位模式。

只有在 A、B 两点同时有效时，上位机软件才会执行标定计算，否则如果只有 1 个点标定有效，或者标定点都无效，数学上无法求出 $Y=MX+N$ 的系数 M 和 N，所以上位机软件会忽略标定参数，按照理论值进行计算电压。

上式并未实际计算出 M 和 N，再对数字量进行转换，而是直接一次性计算出来标定值，省略了中间过程。在上式中， $(A2 - A1) / (D2 - D1)$ 的结果即为 M, A1 的值即为 N，data-d1 的目的是计算 $Y=MX+N$ 式子中 X 的值，是把 D1 作为 X 的原点来进行计算的。

通过此函数调用，即可将采集卡上传的数字量 data 线性化换算成对应的物理量。

☒ 启用标定

☐ 电压型

☒ ADC量程10V

模拟输入

☒ ADC_1 3.00077 V

☒ ADC_2 2.95350 V

如上图所示，将采集卡的通道 1 进行 2 点标定后，将 DAC 的输出电压设置成 3V，可以看到通道 1 的值很接近 3V，通道 2 的值误差明显较大。此时 DAC 的输出同时接入 ADC 通道 1 和通道 2，此时 ADC 的值理论上应该非常接近。DAC 精度较差，实测 ADC1 的数值较准确，即为标定的效果已经实现。

如果不需要再进行标定，只需要在标定模式下，点击对应通道的清除标定按钮，对应通道的所有标定参数以及有效标志都会清除。

点击清除前：

清除标定

清除标定

标定点A

状态

数字量

模拟量

☒ 有效

10042332

2.00000

标定点

☐ 有效

0

0.00000

标定点

标定点B

状态

数字量

模拟量

☒ 有效

11688477

4.00000

标定点

☐ 有效

0

0.00000

标定点

点击清除后：

清除标定

清除标定

状态

数字量

模拟量

☐ 有效

0

0.00000

标定点

☐ 有效

0

0.00000

标定点

状态

数字量

模拟量

☐ 有效

0

0.00000

标定点

☐ 有效

0

0.00000

标定点

标定点A

标定点B

10 采集卡 MODBUS RTU 功能

11 采集卡 MODBUS TCP 功能

12 采集卡 USB 通信协议

12.1 C# 上位机接收校验部分代码解析(上位机核心代码)

```

1 个引用
private void rxTim_Tick(object sender, EventArgs e)
{
    //检索收到的数据开头是否为0x55aa, 如果不是则继续轮循下一步
    while (usbRxMsgBuf.Count > 5)
    {
        //找0x55作为包头
        bool findNewHead = false;
        byte[] usbRxBuf = usbRxMsgBuf.ToArray();

        for (UInt16 i = 0; i < usbRxBuf.Length; i++)
        {
            if (usbRxBuf[i] == 0x55 && i != (usbRxBuf.Length - 1))
            {
                if (usbRxBuf[i + 1] == 0xaa)
                {
                    if (i > 0 && usbRxMsgBuf.Count >= i)
                    {
                        //清除源文件数据
                        usbRxMsgBuf.RemoveRange(0, i);
                        break;
                    }

                    //检索到报文开头, 判断是否完成一帧
                    findNewHead = true;
                    msgHead_pRxHead = new msgHead();
                    _pRxHead = (msgHead)msgStruct.BytesToDataStruct(usbRxBuf, typeof(msgHead));

                    if ((usbRxBuf.Length - i) >= SWAP16(_pRxHead.frameLen))
                    {
                        string crcResult = crcCheck.checkCrc16WithTail(usbRxBuf, SWAP16(_pRxHead.frameLen));

                        if (crcResult != "OK")
                        {
                            //校验失败, 说明0x55aa不是包头 或者数据被污染, 重新找包头
                            usbRxMsgBuf.RemoveRange(0, 2);
                            break;
                        }
                        _deviceRxMsg(usbRxMsgBuf.ToArray());
                        usbRxMsgBuf.RemoveRange(0, SWAP16(_pRxHead.frameLen));
                        break;
                    }
                    else
                    {
                        //长度不够, 退出中断继续接收
                        return;
                    }
                }
            }
        }

        //未找到 newHead, 则清空缓存区
        if (!findNewHead)
        {
            usbRxMsgBuf.Clear();
        }
    }
}

```

一、核心定位

这段代码是 WinForm 串口数据帧解析核心逻辑

- 运行在 100ms 定时器 `rxTim` 中
- 作用：从串口数据流中，自动查找有效帧、校验帧、提取完整帧、抛给业务层处理

- 通信协议：

0x55 0xAA + 命令 ID + 帧长度 + 数据段 + CRC16

二、整体流程:每 100ms 检查一次接收缓冲区

1. 找帧头 `0x55 0xAA`
2. 判断一帧数据是否收完整
3. CRC 校验是否正确
4. 正确 → 交给业务层处理，然后删除这一帧
5. 错误 → 丢弃脏数据，重新找帧头

三、逐行解析

```
while (usbRxMsgBuf.Count > 5)
```

- 只有当缓存区字节数 >5 才进入解析
- 因为一帧数据最少也有 6 个字节（帧头 2 + 命令 1 + 长度 2 + CRC2）
- 避免数据不足时无效解析

```
bool findNewHead = false;
```

```
byte[] usbRxBuf = usbRxMsgBuf.ToArray();
```

- `findNewHead`：标记是否找到有效帧头 0x55AA
- 将缓存列表转成数组，方便遍历查找

```
for (UInt16 i = 0; i < usbRxBuf.Length; i++)
```

```
{
    if (usbRxBuf[i] == 0x55 && i != (usbRxBuf.Length - 1))
    {
        if (usbRxBuf[i + 1] == 0xaa)
        {
```

遍历所有数据，查找帧头 0x55 0xAA

这是整个协议解析的起点。

```
if (i > 0 && usbRxMsgBuf.Count >= i)
```

```
{
    usbRxMsgBuf.RemoveRange(0, i);
    break;
}
```

- 如果帧头不在第 0 位，说明前面是脏数据
- 直接删除脏数据，把帧头对齐到缓冲区开头

```
findNewHead = true;
msgHead _pRxHead = new msgHead();
_pRxHead = (msgHead)msgStruct.BytesToDataStruct(usbRxBuf,
typeof(msgHead));
```

- 找到帧头，标记 `findNewHead = true`
- 将缓冲区数据转成协议头结构体，读取帧长度

```
if ((usbRxBuf.Length - i) >= SWAP16(_pRxHead.frameLen))
```

判断：当前缓冲区剩余数据 $\geq$ 一帧完整长度

- 足够 $\rightarrow$ 继续解析
- 不够 $\rightarrow$ 直接 return，等下一次 100ms 再收

```
string crcResult = crcCheck.checkCrc16WithTail(usbRxBuf,
SWAP16(_pRxHead.frameLen));
if (crcResult != "OK")
{
    usbRxMsgBuf.RemoveRange(0, 2);
    break;
}
```

CRC16 校验

- 校验正确 $\rightarrow$ 继续
- 校验错误 $\rightarrow$ 删掉这个错误帧头（0x55AA），重新查找

```
_deviceRxMsg(usbRxMsgBuf.ToArray());
```

将完整正确的一帧数据抛给业务层处理

比如 ADC 数据、DIO 数据、DAC 数据、Modbus 数据等。

```
usbRxMsgBuf.RemoveRange(0, SWAP16(_pRxHead.frameLen));
break;
```

删除已处理完的一帧数据

缓冲区继续保留未处理的后续数据。

```
else
{
    return;
}
```

数据长度不够一帧，直接退出，等待下一个 100ms。

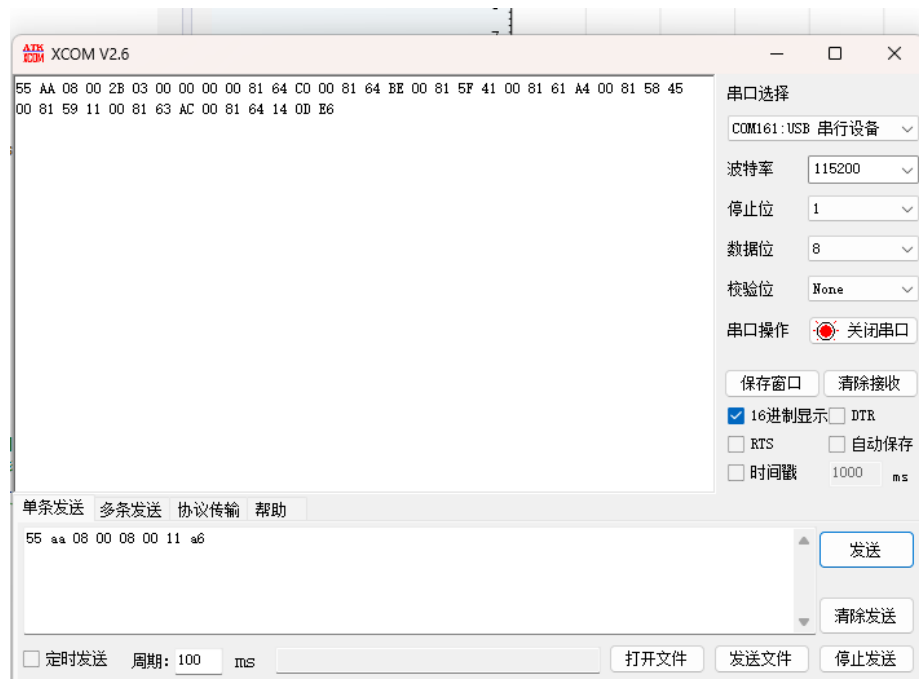
```
if (!findNewHead)
{
    usbRxMsgBuf.Clear();
}
```

遍历完所有数据都没找到 0x55AA

→ 清空缓冲区，避免脏数据堆积。

12.1 单点记录仪

单次读取采集卡数据方式比较简单，虽然也需要 CRC，但是读取过程中发送指令不会变化，所以发送的 CRC 值也是固定的。接收返回的数据也带有 CRC，但是接收解码可以屏蔽掉 CRC 直接使用，如下图所示，发送数据指令为 55 AA 08 00 08 00 11 A6，接收返回的是 55 AA 08 00 2B 03 00 00 00 00 81 64 C0 00 81 64 BE 00 81 5F 41 00 81 61 A4 00 81 58 45 00 81 59 11 00 81 63 AC 00 81 64 14 0D E6



采集卡通信使用的是结构体指针形式来调控数据流，具体解析方法如下：

```

182
183 typedef struct {
184     u16 msgHead;
185     u8 cmdId;
186     u16 frameLen;
187     u8 ch;
188     u16 crc;
189 } msgNoParam;
190

```

数据流为大端模式，55 AA 08 00 08 00 11 A6 中，

55AA 是 msgHead, 代表包头, 用来分割不同的包;

08 代表 cmdId, 为每一帧的指令, 通过 cmdId 来调用不同的接收函数处理数据, 此处 08 代表单点记录仪同时读取 8 通道 adc 和 8 通道 DIN 状态;

00 08 代表帧长度为 8;

00 代表通道号, 此处未用到, 默认即可;

11 A6 代表 CRC 校验值

```

217
218 typedef struct {
219     u16 msgHead; //包头 0x55aa
220     u8 cmdId; //返回指令
221     u16 frameLen; //包长度
222
223     u8 adc_ch_Enable; //8通道adc使能状态
224     u8 adcSignalType;
225     u8 adcRange;
226     u8 din;
227     int32 data[8]; //8个通道的数据 每个通道2个字节
228
229     u16 crc; //crc校验值
230 }msgGetAdcValue;
231

```

接收返回的数据流比较长,但是也是同样的方式来借用结构体解析

55 AA 是 msgHead, 代表包头, 用来分割不同的包;

08 代表 cmdId, 虽然 cmdId 相同但是上位机和下位机调用的函数及结构体不同, 处理方式也不同。

00 2B : 代表帧长度, 为 43 字节

03: 通道使能状态, 上位机处理波形显示时使用, 不影响 8 通道数值。

00: 代表传感器类型, 0 为电压型, 1 为电流型

00: 代表 ADC 量程, 0 为±5V, 1 为±10V。

00: 代表 8 通道数字输入状态, bit0~bit7 分别代表 DIN1~DIN8, 每个通道占用 1 个 Bit。

00 81 64 C0: 通道 1 的 ADC 值, 0x00800000 代表 0V, 24 位形式, 0x00FFFFFF 为最大量程值(5V 或 10V), 0x00000000 为最小量程值(-5V 或-10V)。00 81 64 C0 为 0x008164C0, 换算为 $(0x008164C0 - 0x00800000) / 0x800000 * 5$ 结果为 0.010V

00 81 64 BE: 通道 2 的 ADC 值

00 81 5F 41: 通道 3 的 ADC 值

00 81 61 A4: 通道 4 的 ADC 值

00 81 58 45: 通道 5 的 ADC 值

00 81 59 11: 通道 6 的 ADC 值

00 81 63 AC: 通道 7 的 ADC 值

00 81 64 14: 通道 8 的 ADC 值

0D E6 : CRC16 校验值, 可忽略不用

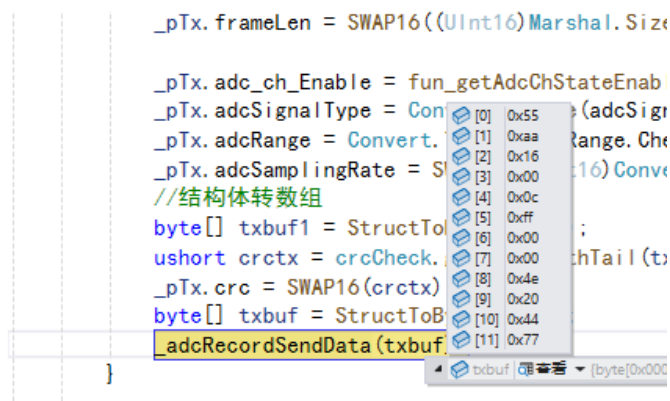
12.2 在线记录仪

ADC 在线记录仪模式功能使用手册

一、功能概述

本代码实现 ADC 多通道在线记录仪核心功能, 采用双缓冲区 (A/B) 交替采集机制, 在「在线记录仪模式」下, 按配置自动采集指定 ADC 通道的采样数据, 缓冲区满后自动切换, 为数据上传 / 存储提供稳定、连续的采样能力, 适用于实时波形采集、数据监测场景。

如下图所示 16 进制下的数据流为开启在线记录仪的指令

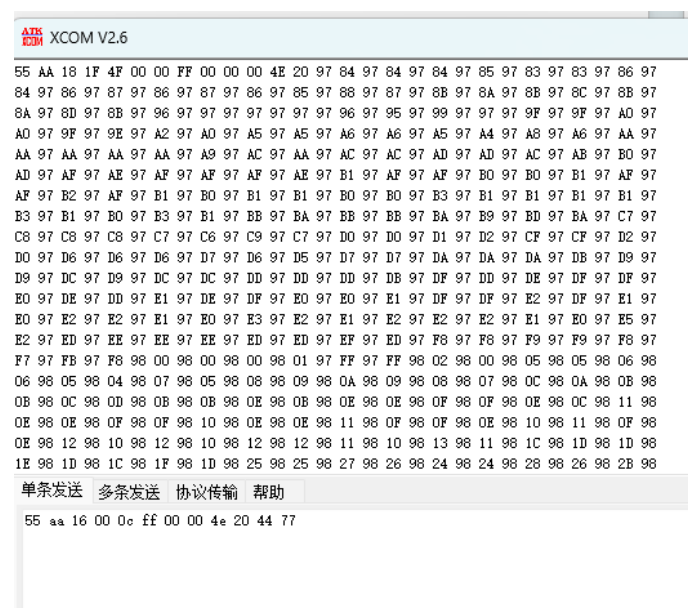


开启的结构体在上位机为

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct msgAdcStartOnlineRecord
{
    public UInt16 msgHead; //0x55aa
    public byte cmdId; //0x16
    public UInt16 frameLen; //0x000c
    public byte adc_ch_Enable; //0xff
    public byte adcSignalType; //0x0
    public byte adcRange; //0x0
    public UInt16 adcSamplingRate; //0x4e20
    public UInt16 crc; //0x4477
}
```

发送 55 aa 16 00 0c ff 00 00 4e 20 44 77 指令后，串口会连续不间断的返回 adc 数据，无需上位机再次发送读取指令，采集卡会持续上传数据直到用户手动发送停止命令。

下面截图简单截取了一些返回数据流，可以用来分析报文结构



二、核心数据结构定义

1. ADC 发送缓冲区结构体 _adcSendBuf

存储完整的 ADC 采集数据包，包含帧头、配置、采样数据、校验位，是数据交互的核心载体。

```
typedef struct{
    u16 msgHead;           // 数据帧头（固定标识，用于数据解析同步）
    u8 cmdId;              // 命令 ID（标识当前数据包的指令类型）
    u16 frameLen;          // 整个数据帧总长度（字节）
    u16 dataLen;           // 有效采样数据长度（字节）
    u8 adc_ch_Enable;      // ADC 8 通道使能状态（bit0~bit7 对应通道 0~7，1=使能，0=禁用）
    u8 adcSignalType;      // ADC 信号类型配置（如电压/电流、模拟量类型）
    u8 adcRange;           // ADC 量程配置（如±5V、0~10V 等测量范围）
    u8 adcStartCh;         // 采样起始通道（第一个被使能的 ADC 通道号）
    u16 samplingRate;      // ADC 采样率（单位：SPS，每秒采样点数）
    u16 data[ADC_WAVE_ONCE_LEN]; // 采样数据缓冲区（长度=ADC_WAVE_ONCE_LEN，固定 4000 点）
    u16 crc;               // 数据校验位 CRC16
}_adcSendBuf;
```

以起始字节 55 AA 18 1F 4F 00 00 FF 00 00 00 4E 20 97 84 97 84 97 84……为例

0X55AA 为包头，

0X18 为 cmdId，

0x1f4f 为包长度，

换算 10 进制为 8015，

0x0000 为数据长度，在线记录仪每次都是满帧发送，所以此处未用到。

0xff 为 adc_ch_Enable

再往后 3 个 00 分别为传感器类型（电压为 0 电流为 1）、电压量程（±5V 为 0，±10V 为 1），起始通道

0x4e20 为采样率，换算为 20K

再往后 0x9784 为通道电压，持续循环 4000 次 最后结尾是 CRC

数据流非常快，人眼无法看清，只能通过上位机解析


XCOM V2.6

55 AA 19 0F B0 00 00 33 F0 01 FF 00 00 00 66 89 67 04 67 63 67 E4 68 34 68 AD 69 14
69 85 69 F7 6A 5C 6A D9 6B 3F 6B BD 6C 21 6C 9A 6D 08 6D 75 6D F0 6E 55 6E D7 6F 49
6F BB 70 22 70 94 71 0F 71 73 71 F4 72 5A 72 E0 73 4B 73 C7 74 35 74 A3 75 26 75 89
76 12 76 84 77 03 77 75 77 D8 78 53 78 B4 79 40 79 AB 7A 2A 7A 9C 7B 1C 7B 88 7B F9
7C 7E 7C E2 7D 6B 7D DB 7E 5A 7E CE 7F 43 7F B5 80 29 80 AA 81 11 81 96 82 05 82 84
82 F9 83 67 83 E2 84 4D 84 DA 85 3E 85 C5 86 32 86 AE 87 23 87 8E 88 0F 88 75 88 FC
89 63 89 E7 8A 57 8A CC 8B 40 8B A3 8C 2F 8C 96 8D 14 8D 86 8D FC 8E 73 8E E1 8F 5C
8F C6 90 48 90 B0 91 26 91 96 92 0B 92 80 92 E0 93 61 93 BF 94 3D 94 AA 95 23 95 91
95 F9 96 7A 96 D9 97 59 97 B9 98 31 98 97 99 03 99 7B 99 D2 9A 53 9A AB 9B 22 9B 82
9B F7 9C 62 9C C3 9D 37 9D 96 9E 0D 9E 6F 9E DC 9F 3E 9F A3 A0 0D A0 66 A0 DA A1 31
A1 A4 A1 FC A2 69 A2 C6 A3 1F A3 8C A3 E2 A4 4F A4 A3 A5 10 A5 6C A5 CD A6 33 A6 8F
A6 ED A7 42 A7 A8 A7 FC A8 58 A8 B1 A9 0F A9 69 A9 B5 AA 1F AA 6A AA C3 AB 16 AB 6B
AB C6 AC 14 AC 6D AC B7 AD 1E AD 63 AD BB AE 0A AE 61 AE AD AE EF AF 47 AF 8A AF E0
B0 26 B0 77 B0 B7 B1 01 B1 50 B1 94 B1 E0 B2 22 B2 6B B2 AC B2 F0 B3 38 B3 70 B3 BE
B3 FD B4 46 B4 80 B4 C1 B5 01 B5 37 B5 80 B5 B4 B5 FB B6 35 B6 70 66 89 67 04 67 63
67 E6 68 34 68 AD 69 14 69 85 69 F7 6A 5B 6A D9 6B 3C 6B BE 6C 21 6C 9B 6D 06 6D 76
6D F0 6E 54 6E D7 6F 47 6F BB 70 21 70 95 71 0F 71 73 71 F5 72 5A 72 E0 73 4B 73 C8
74 36 74 A3 75 26 75 8A 76 12 76 84 77 03 77 76 77 D7 78 53 78 B5 79 40 79 A9 7A 29
7A 9B 7B 1C 7B 89 7B F9 7C 7D 7C E3 7D 6A 7D DB 7E 5A 7E CD 7F 43 7F B5 80 29 80 A9
81 10 81 96 82 05 82 85 82 F8 83 66 83 E2 84 4C 84 DA 85 3E 85 C4 86 31 86 AF 87 23

单条发送 多条发送 协议传输 帮助

55 aa 19 00 08 00 14 9a

☐ 定时发送 周期: 100 ms 打开文件
☒ 16进制发送 ☐ 发送新行 0% [【火爆全网】](#)